



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Stanislav Baláž
March 12, 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusions
- Appendix

[SpaceX-Landing-Prediction-Technical-Enhancements](#)

Executive Summary

Objective: Build a model to predict SpaceX Falcon 9 first-stage landing success to support competitive bidding

Methodologies:

- Data was collected from SpaceX API and Wikipedia using Python Requests
- Data wrangling, preprocessing and analysis performed with Pandas, SQL, Folium, Plotly Dash and Scikit-Learn => 83 features used to visualize patterns and predict landing success

Results:

- Increasing trend in success rate from 2013 to 2020
- High success rates uncovered in SSO, VLEO orbits, payload range 3000 – 4000 kg and booster version FT
- Logistic Regression outperformed other evaluated models with Accuracy and Generalization Capacity (Train/Test gap)

Final Model Evaluation Metrics

- Accuracy 83.3%
- Recall 100%
- Precision 80%
- Specificity 50%

Action

Tune the model to improve prediction of landing failures

Introduction

Background and Context

- SpaceX is a private American aerospace company established with a vision of decreasing the costs of space launches
- Most of savings is linked to possibility of reusing the first stage, which makes SpaceX highly competitive

Objectives

- Predicting whether the first stage will land successfully or not, helps calculate estimated launch costs => competitive offers to potential customers
- The objective of this project is to build a prediction model which allows us to answer the key question: **Will the first stage land successfully or not?**



Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology: data was collected via SpaceX REST API and webscraping Wikipedia List of Falcon 9 and Falcon Heavy launches
- Performed data wrangling: collected data was processed using Pandas to retain relevant records, handle missing values, and create labels for machine learning
- Performed exploratory data analysis (EDA) using visualization and SQL
- Performed interactive visual analytics using Folium and Plotly Dash
- Performed predictive analysis using classification models: several classification models were built, tuned and evaluated. The best model has been identified.

Data Collection

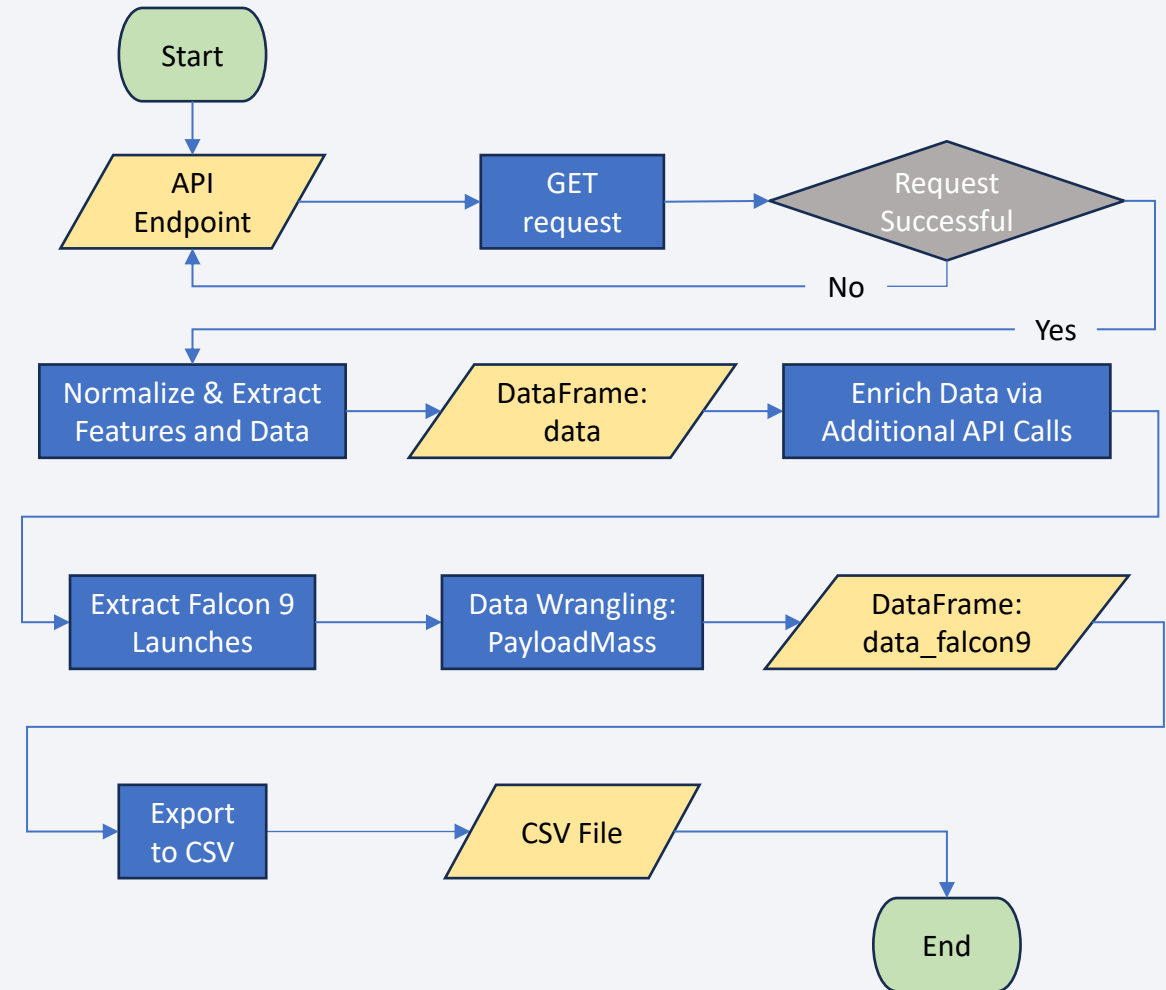
Data was collected via two sources:

- SpaceX REST API:
 - Original data source: URL = <https://api.spacexdata.com/v4/launches/past>
 - Data Source actually used for consistency reasons: URL = https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/API_call_spacex_api.json
 - Data was enriched from secondary API endpoints (rockets, launchpads, payloads, and cores), URL = <https://api.spacexdata.com/v4/item>
- Wikipedia List of Falcon 9 and Falcon Heavy launches: URL = https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922

Data retrieved via SpaceX REST API was used for further modelling

Data Collection – SpaceX API

- Data retrieval from SpaceX REST API using Python requests library
- Data normalization and processing performed via Pandas DataFrames
- Specific features extracted and utilized to enrich data using secondary SpaceX API endpoints (via auxiliary functions)
- Records up to November 13, 2020 with one core and only related to Falcon 9 were retained
- Data wrangling – missing PayloadMass values filled with mean
- Data export to CSV file (dataset_part_1.csv)

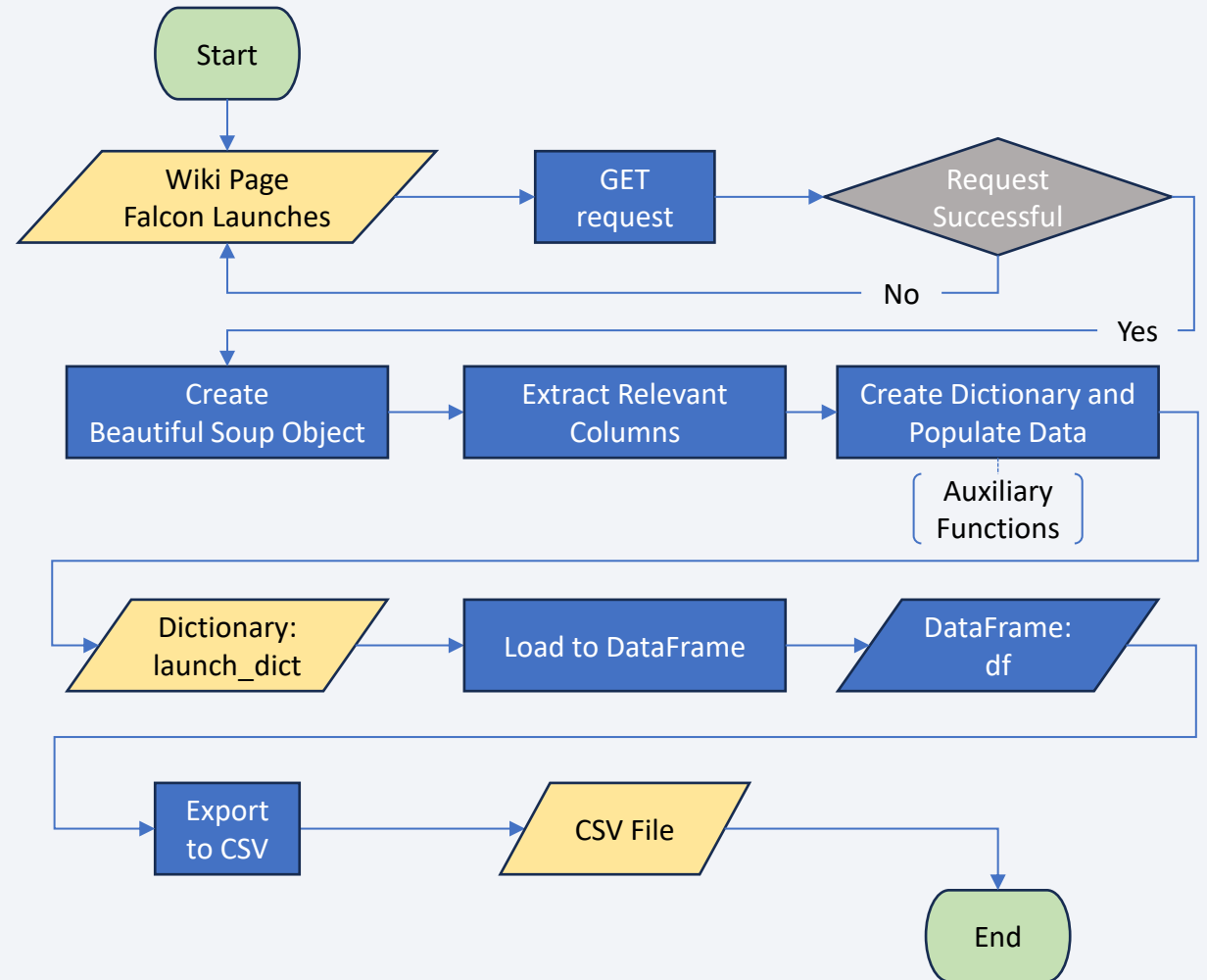


[GitHub Link to API Notebook](#)

Data Collection - Scraping

- Data scraped from Wiki page with List of Falcon 9 and Falcon Heavy launches using Python requests and BeautifulSoup libraries
- Data extracted from BeautifulSoup object and preprocessed using auxiliary functions => dictionary launch_dict
- Data loaded to DataFrame df
- Data export to CSV file (spacex_web_scraped.csv)

[GitHub Link to Scraping Notebook](#)

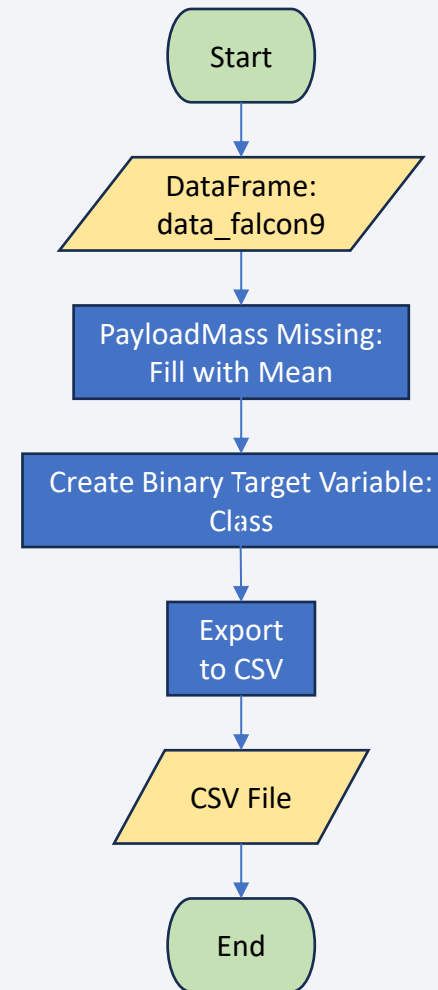


Data Wrangling

- Missing values for PayloadMass were filled in by mean value (API Notebook)
- Class, binary target variable for machine learning was derived from categorical Outcome (Landing Outcome) variable (Data Wrangling Notebook)
- Data export to CSV file (dataset_part_2.csv)

[GitHub Link to API Notebook](#) – see Task 3: Dealing with Missing Values

[GitHub Link to Data Wrangling Notebook](#) – see TASK 4: Create a landing outcome label from Outcome column



EDA with Data Visualization

EDA visualization contains the following charts:

- **Categorical scatter plots** – utilized to visualize distribution and density of landing successes for multivariate relationships (Flight Number vs Payload Mass/ Launch Site/ Orbit, Payload Mass vs Launch Site/ Orbit)
- **Bar Chart of Success Rates by Orbit Type** – used to identify the highest and lowest performing orbits in terms of landing success rate (along with the number of launches for statistical context)
- **Line Chart of Success Rate Evolution** – visualizing the increasing trend in yearly success rates evolution (along with the number of launches for statistical context)

[GitHub Link to EDA Visualization Notebook](#)

EDA with SQL

SQL queries performed using SQL Cell Magic

- Data table preparation – SPACEXTABLE created with data from DataFrame df
- Launch Sites Summary:
 - List of analyzed Launch Sites created
 - Records for sites starting with CCA retrieved to provide brief overview
- Customer Analytics – total Payload Mass carried for NASA (CRS) aggregated

[GitHub Link to EDA with SQL](#)

EDA with SQL

SQL queries performed using SQL Cell Magic

- Booster Version Analytics:
 - Average Payload Mass carried by F9 v1.1 has been calculated
 - Overview of boosters carrying 4000 kg to 6000 kg Payload Mass with landing success in drone ship
 - Summary of boosters carrying maximum Payload Mass for the analyzed period of time

[GitHub Link to EDA with SQL](#)

EDA with SQL

SQL queries performed using SQL Cell Magic

- Landing Outcome Evaluation:
 - Date of the first successful landing in ground pad identified
 - Boosters with failure in drone ship landing in 2015 identified
 - Aggregation of landing outcomes in the years 2010 to 2017
- Mission Outcome Analysis – aggregated successes and failures of the analyzed missions

[GitHub Link to EDA with SQL](#)

Build an Interactive Map with Folium

Folium map was used to capture locations of SpaceX Launch Sites

Various **map objects** were utilized to highlight selected properties:

- Circles – identify exact location of each launch site with 1 km radius
- Markers – added to the map to visualize:
 - Names of Launch Sites
 - Individual launches and their landing outcome (green – success, red – failure)
 - Distance from the Launch Sites to closest points in their proximities (coastline, city, highway, railway)
- Marker Cluster – utilized to simplify the map by dynamically grouping individual launch markers, since many of them share the same coordinates
- Lines – utilized as a trace to closest points in the proximities of Launch Sites

[GitHub Link to Interactive Map with Folium](#)

[Nbviewer Link to Interactive Map with Folium](#)

Build a Dashboard with Plotly Dash

SpaceX Launch Records Dashboard contains:

- Dropdown Box – provides option to select All Sites for the overview of all launches or a specific Launch Site for the analysis of a specific site
- Range Slider – possibility to select a certain payload range to be analyzed
- Pie Chart – visualizes success rates of all sites if All Sites are selected in a dropdown. If a specific site is picked in a dropdown, then success and failures portions of the site is presented
- Scatter chart – utilized to analyze correlation between Success, Payload and Booster Version Category. Represent all sites or a specific site, depending on the Dropdown Box selection. Payload range selected in the Range Slider is rendered.

The components and their interactions provides flexible visual analytics on SpaceX Launch Success of selected site / all sites.

Dashboard also presents success of individual launches for a selectable Payload range, all this with indication of the respective Booster Version Category.

[GitHub Link to Dashboard with Plotly Dash](#)

Predictive Analysis (Classification)

- Feature engineering – performed one-hot encoding of categorical variables (EDA with Data Visualization notebook) prior to predictive analysis
- Used libraries: Pandas, NumPy, Matplotlib, Seaborn, Scikit-Learn
- Features were loaded to variable X (83 columns), Target to variable Y
- Features were standardized using `StandardScaler()`
- Both X and Y were split to training and testing data via `train_test_split()` with stratification
- Classification models using Logistic Regression, Support Vector Machine, Decision Tree, and k-Nearest Neighbors were built, improved, and evaluated

[GitHub Link to Predictive Analysis](#)

Predictive Analysis (Classification)

The same process was applied to each of the Classification models

- Define Hyperparameter Space – build the parameters grid to be used for hyperparameters tuning (model improvement)
- Define the Estimator – build the classification model to be used
- Define GridSearchCV object – build the object for hyperparameter tuning with defined Estimator and Hyperparameter Space, using 10-fold cross validation and accuracy as the default classification scoring metric
- Fit/Train GridSearchCV – improve the model using train data and find the best combination of hyperparameters
- Evaluate the model using test data with `.score( )` method and confusion matrix

[GitHub Link to Predictive Analysis](#)

Predictive Analysis (Classification)

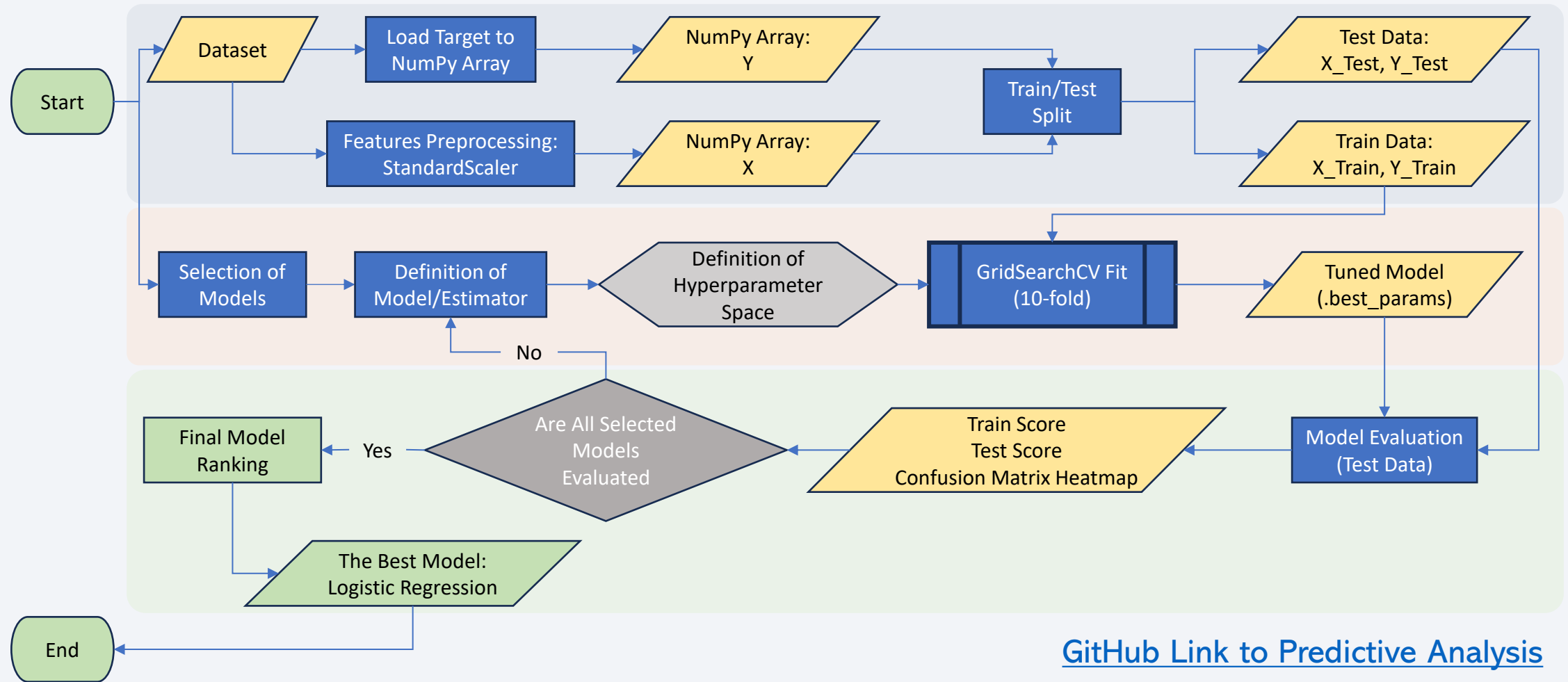
The approach to identify the best classification performing model

- Metric: Accuracy
- Visual Analytics: Confusion Matrix
- Models were sorted in descending order of their evaluation results (test data)
- The highest accuracy of 83.3% was reached by Logistic Regression and Support Vector Machine models
- Then the model with smaller gap between test and train accuracy, i.e. indicating better generalization was selected as the best performing model

Logistic Regression

[GitHub Link to Predictive Analysis](#)

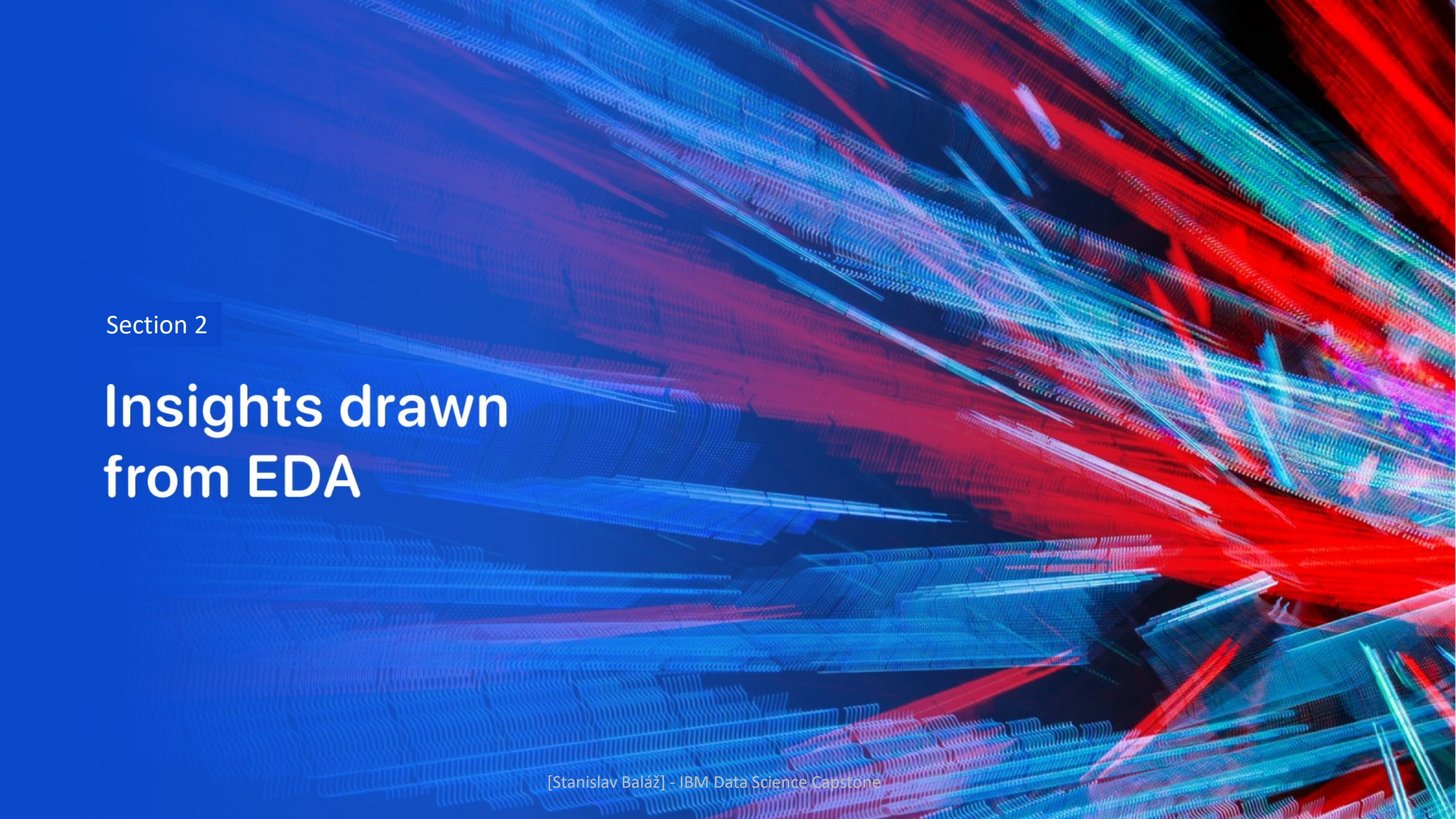
Predictive Analysis (Classification)



[GitHub Link to Predictive Analysis](#)

Results

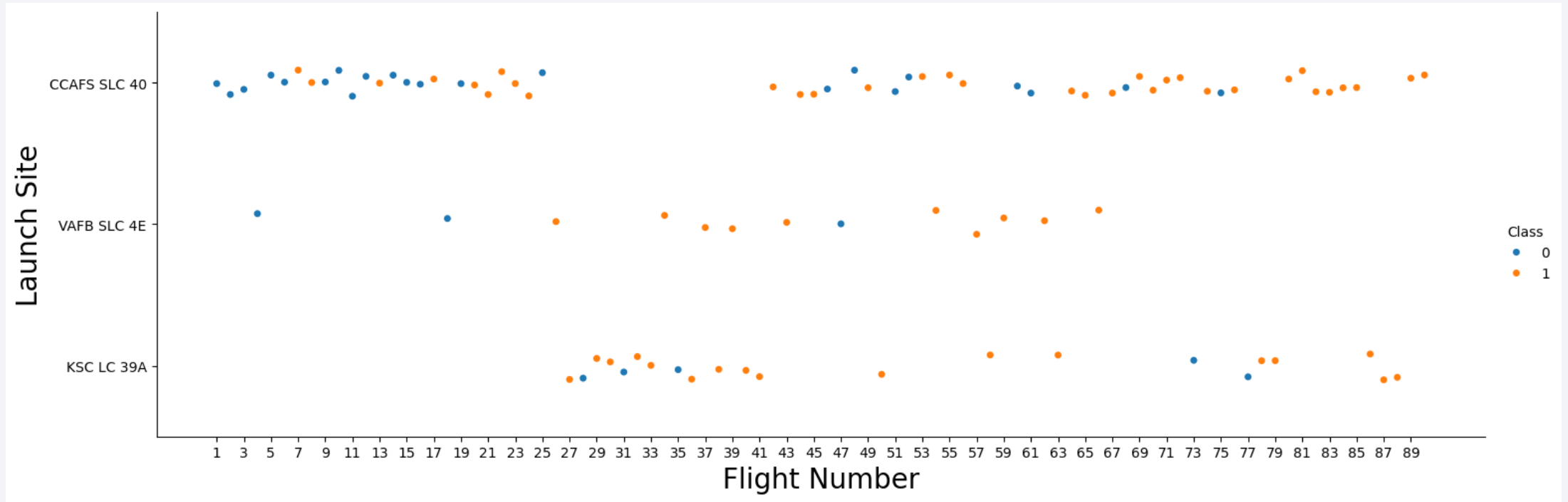
- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results



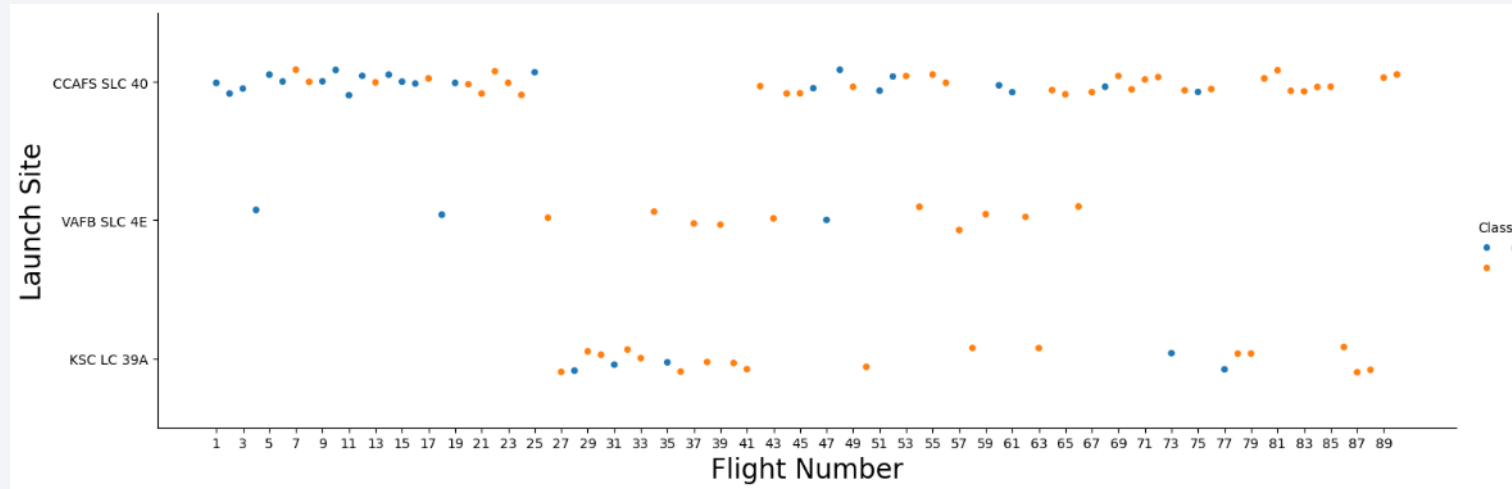
Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

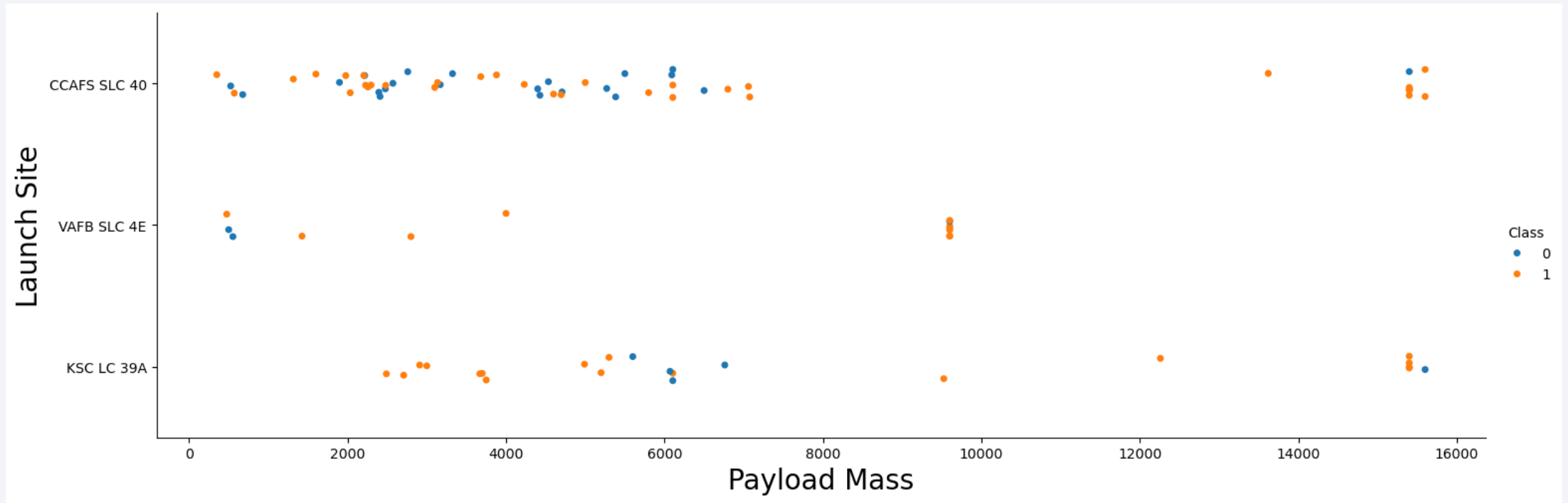


Flight Number vs. Launch Site

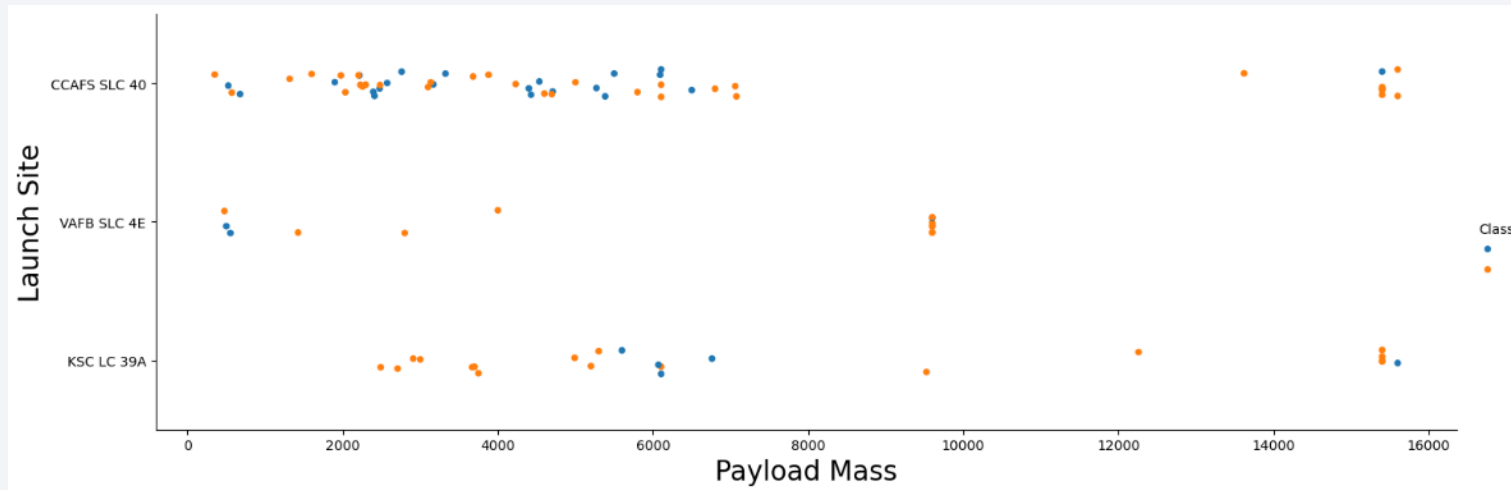


- First set of flights (25) had a low rate of successful returns of stage 1. Most of them were launched from CCAFS SLC 40.
- For later launches (starting from #26), overall portion of successful returns increased.
 - VAFB SLC 4E has the highest rate of successful returns in later flights
 - CCAFS SLC 40 and KSLC 39A shows similar successful returns ratio in later flights
 - CCAFS SLC 40 successful returns ratio improved during the last 20 flights

Payload vs. Launch Site

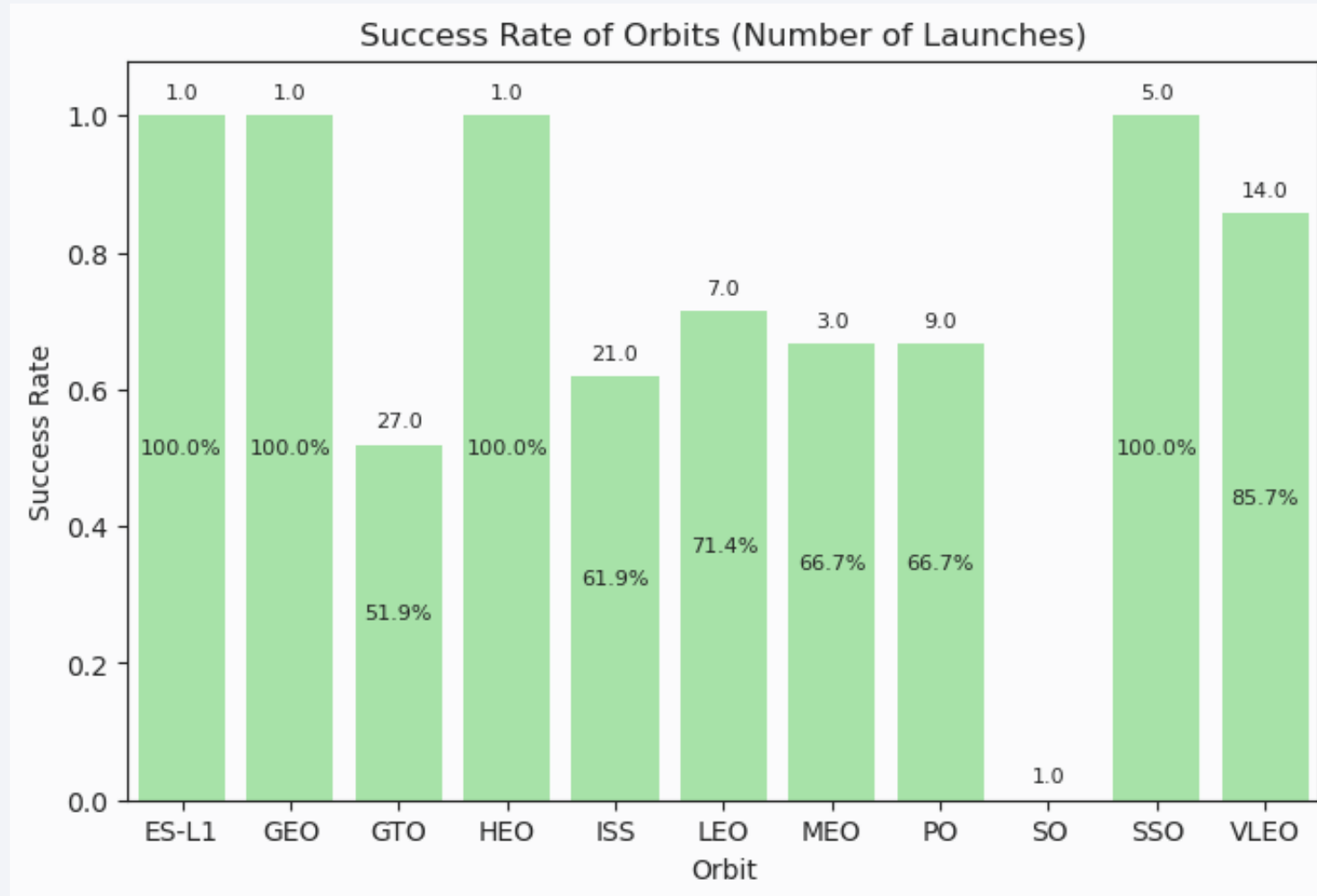


Payload vs. Launch Site

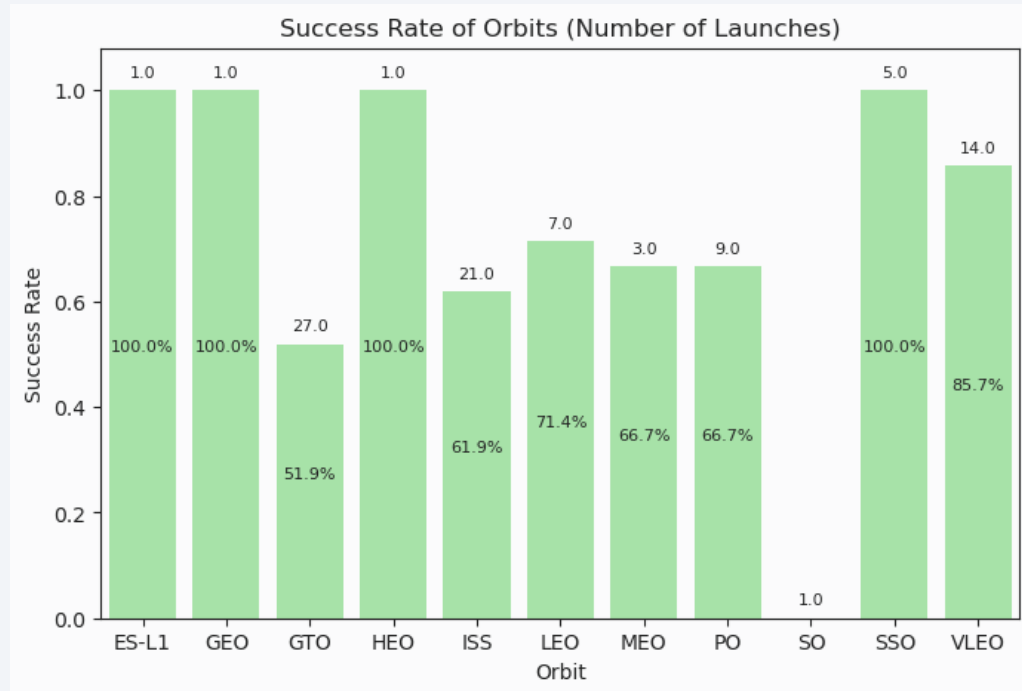


- VAFB SLC 4E – maximum launched payload mass 10000 kg
- Almost all Payloads 10000 kg (+/- 10%) are launched from VAFB SLC 4E

Success Rate vs. Orbit Type

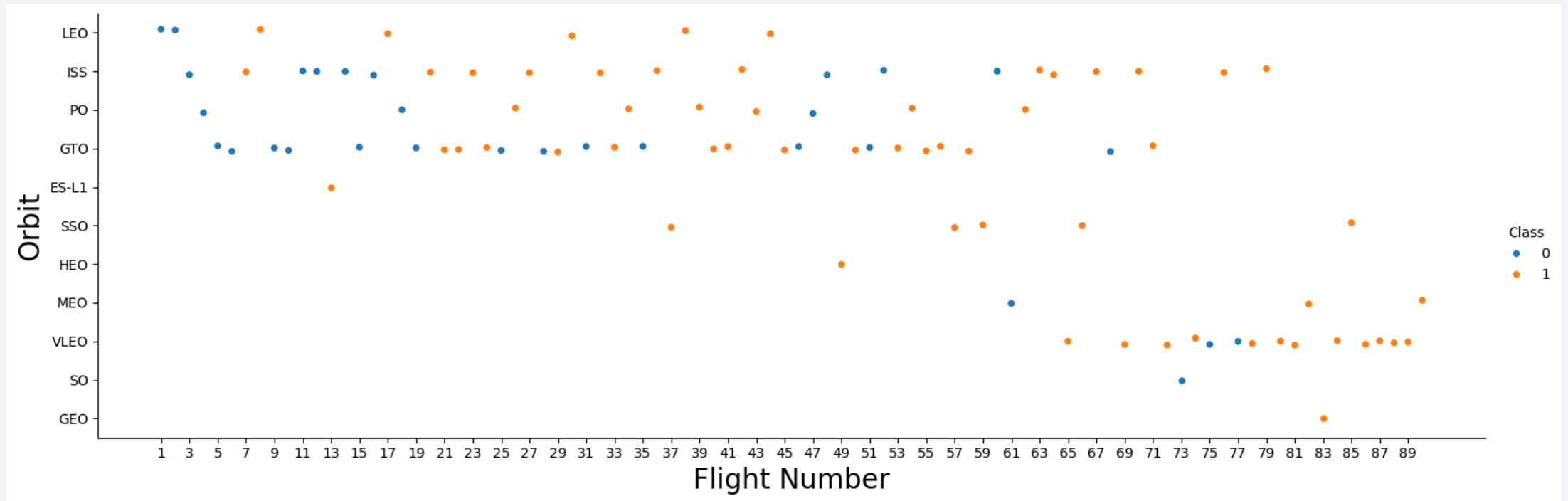


Success Rate vs. Orbit Type

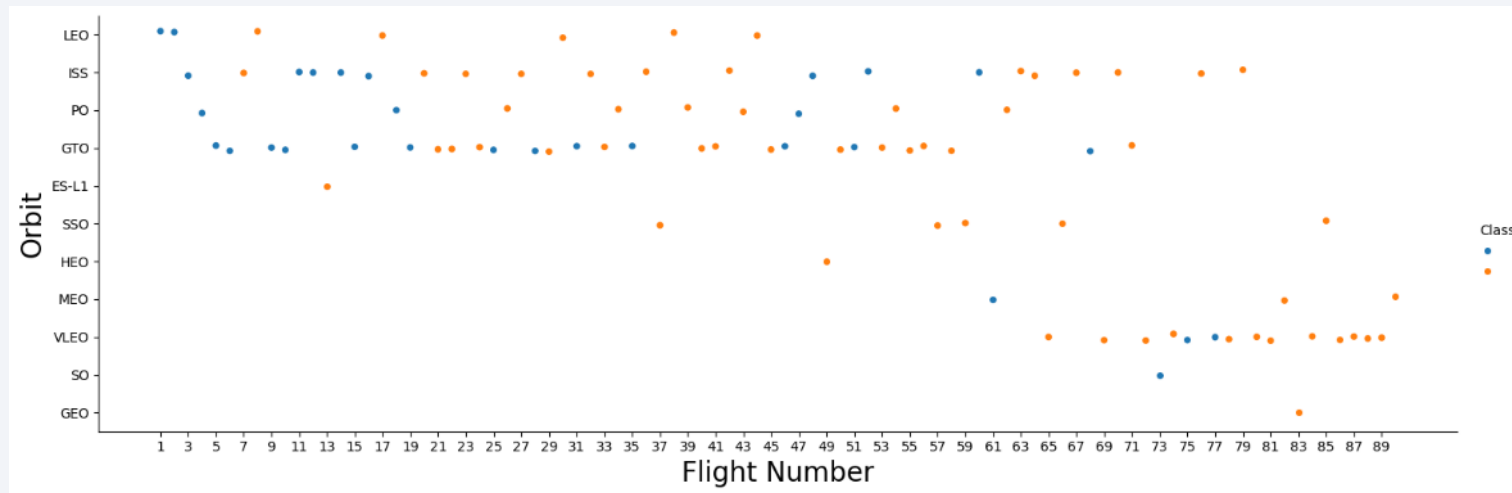


- SSO orbit success rate = 100%, with 5 launches
- VLEO shows good rate of 86% for 14 launches
- Orbits ES-L1, GEO, HEO – 100% success rate. Only one launch is available for each => low statistical significance

Flight Number vs. Orbit Type

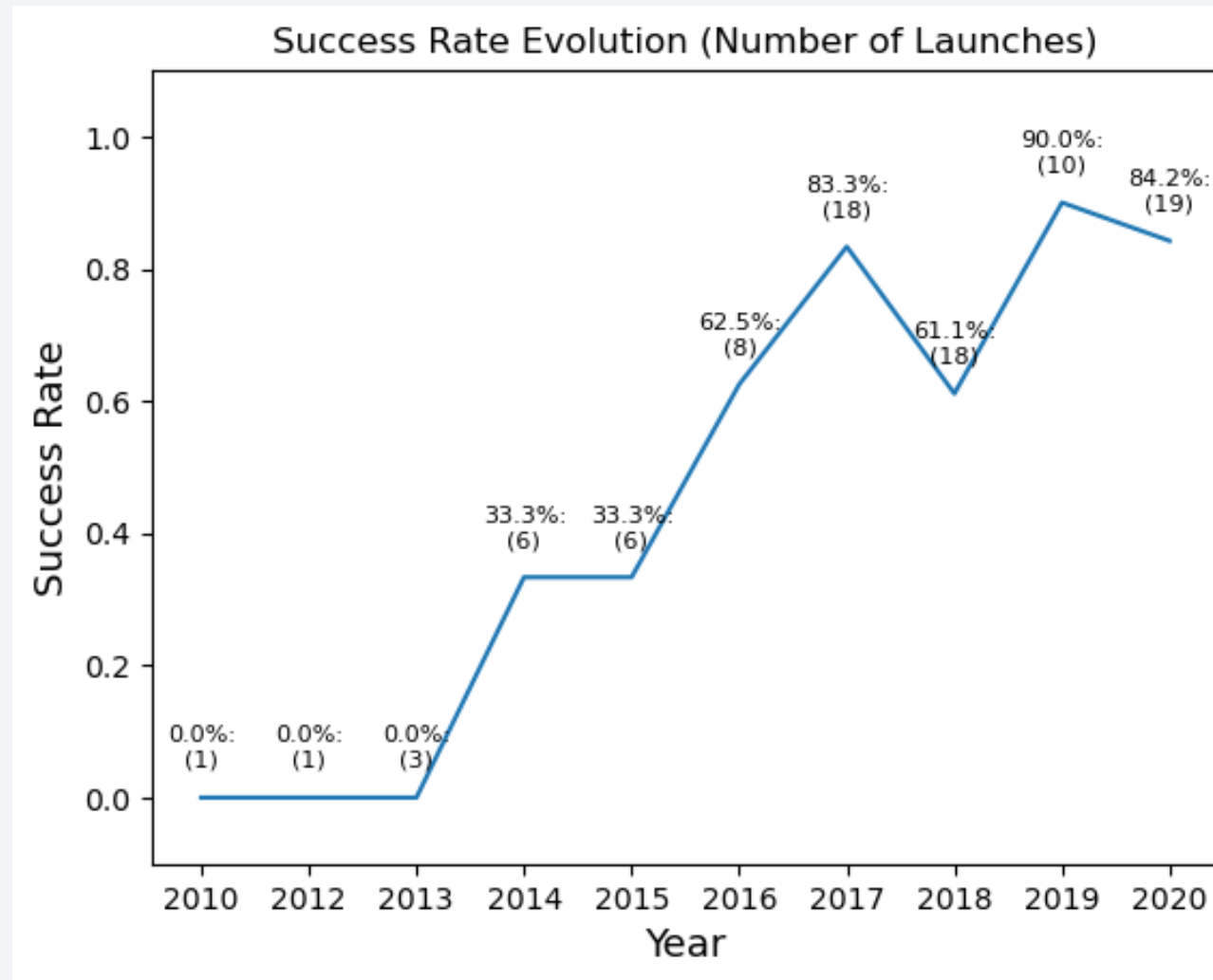


Payload vs. Orbit Type

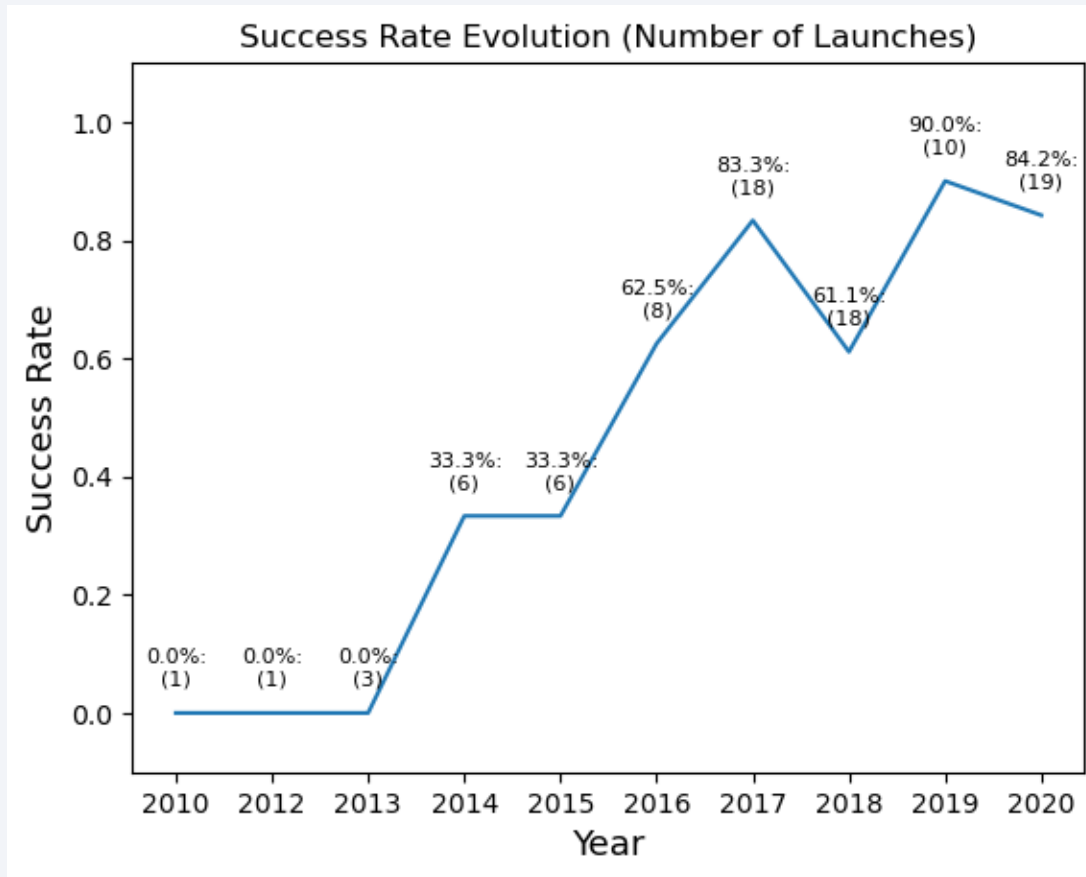


- LEO – success seems to be related to the number of flights
- GTO – there appears to be no relationship between flight number and success
- VLEO – high success rate, the first launch was #65 (June 2019) – start of Starlink era

Launch Success Yearly Trend



Launch Success Yearly Trend



- Success rate upward trend in the analyzed period since 2013 to 2020 with local dips
- Local dips in 2018 and 2020

All Launch Site Names

```
%%sql  
select distinct "Launch_Site" from SPACEXTABLE
```

```
* sqlite:///my_data1.db  
Done.
```

Launch_Site

CCAFS LC-40

VAFB SLC-4E

KSC LC-39A

CCAFS SLC-40

- A SELECT statement with DISTINCT clause was utilized to eliminate duplicities from the result
- There are four Launch Sites in the analyzed data

Launch Site Names Begin with 'CCA'

```
%%sql
select * from SPACEXTABLE where "Launch_Site" like "CCA%" limit 5
```

```
* sqlite:///my_data1.db
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

- A SELECT statement with WHERE clause containing operator LIKE and using the pattern with wild character % was utilized to pull records with Launch_Site starting with “CCA”. Keyword LIMIT with literal 5 was utilized to restrict number of returned records to 5
- The result shows exactly 5 records, all beginning with “CCA”

Total Payload Mass

```
%%sql
select "Customer", sum(PAYLOAD_MASS_KG_) as Total_Payload_Mass from SPACE_TABLE where "Customer" = "NASA (CRS)"
* sqlite:///my_data1.db
Done.
```

Customer	Total_Payload_Mass
NASA (CRS)	45596

- A SELECT statement with SUM function and WHERE clause was utilized to pull records with Customer “NASA (CRS)” and summarize the Payload Mass.
- The result shows that a cumulative Payload Mass of 45596 kg was carried for NASA (CRS)

Average Payload Mass by F9 v1.1

```
%%sql
select avg(PAYLOAD_MASS__KG_) as Average_Payload_Mass from SPACEXTABLE where "Booster_Version" like "F9 v1.1%"

* sqlite:///my_data1.db
Done.
```

Average_Payload_Mass
2534.6666666666665

- A SELECT statement with AVG function and WHERE clause containing operator LIKE and using the pattern with wild character % was utilized to pull records with Booster_Version starting with “F9 v1.1”.
- The result shows that an average Payload Mass carried by the Booster_Version F9 v1.1 amounted to 2535 kg

First Successful Ground Landing Date

```
%%sql
select min(Date) as Date from SPACEXTABLE where "Landing_Outcome" = "Success (ground pad)"

* sqlite:///my_data1.db
Done.
```

Date
2015-12-22

- A SELECT statement, with MIN function and WHERE clause was utilized to identify the date of the first successful ground landing
- Using ISO 8601 date format allowed for direct application of the MIN function
- The first successful ground landing was performed on December 22, 2015

Successful Drone Ship Landing with Payload between 4000 and 6000

```
%%sql
select Booster_Version, PAYLOAD_MASS_KG_ from SPACEXTABLE
  where "Landing_Outcome" = "Success (drone ship)" and
        PAYLOAD_MASS_KG_ > 4000 and
        PAYLOAD_MASS_KG_ < 6000
```

```
* sqlite:///my_data1.db
Done.
```

Booster_Version	PAYLOAD_MASS_KG_
F9 FT B1022	4696
F9 FT B1026	4600
F9 FT B1021.2	5300
F9 FT B1031.2	5200

- A SELECT statement with WHERE clause using operators AND was utilized to identify successful drone ship landings with Payloads between 4000 kg and 6000 kg
- Four boosters were identified as meeting the specified criteria

Total Number of Successful and Failure Mission Outcomes

Version 1

```
%%sql
select "Mission_Outcome", count(*) as Count from SPACEXTABLE group by "Mission_Outcome"

* sqlite:///my_data1.db
Done.
```

Mission_Outcome	Count
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Version 2

```
%%sql
-- Version 2
select substr("Mission_Outcome", 1, 7) as Mission_Outcome, count(*) as Count
from SPACEXTABLE group by substr("Mission_Outcome", 1, 7)

* sqlite:///my_data1.db
Done.
```

Mission_Outcome	Count
Failure	1
Success	100

- Two versions of a query were designed. **Version 2** is considered the final version.
- A SELECT statement using SUBSTR and COUNT functions with GROUP BY clause was utilized to aggregate successes and failures in Mission Outcome
- One failure and one hundred success were identified in the analyzed records

Boosters Carried Maximum Payload

```
%%sql
select Booster_Version, PAYLOAD_MASS__KG_ from SPACE_TABLE
where PAYLOAD_MASS__KG_ =
    (select max(PAYLOAD_MASS__KG_) from SPACE_TABLE)
order by Booster_Version asc
```

```
* sqlite:///my_data1.db
Done.
```

Booster_Version	PAYLOAD_MASS__KG_
F9 B5 B1048.4	15600
F9 B5 B1048.5	15600
F9 B5 B1049.4	15600
F9 B5 B1049.5	15600
F9 B5 B1049.7	15600
F9 B5 B1051.3	15600
F9 B5 B1051.4	15600
F9 B5 B1051.6	15600
F9 B5 B1056.4	15600
F9 B5 B1058.3	15600
F9 B5 B1060.2	15600
F9 B5 B1060.3	15600

- A SELECT statement with WHERE clause using subquery with MAX function, and with ORDER BY clause was utilized to identify the Booster Versions which carried the maximum Payload Mass
- Maximum Payload Mass of 15600 kg was carried by twelve Booster Versions
- The result was sorted by Booster Version in ascending order

2015 Launch Records

```
%%sql
select substr(Date, 6, 2) as Month, Landing_Outcome, Booster_Version, Launch_Site
from SPACEXTABLE
where Landing_Outcome = "Failure (drone ship)" and
      substr(Date, 1, 4)='2015'
```

```
* sqlite:///my_data1.db
Done.
```

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

- A SELECT statement with SUBSTR functions and WHERE clause was utilized to identify records with failure Landing Outcome in drone ships in 2015
- Two records have been selected as failure drone ship landings in 2015

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

```
%%sql
select
    Landing_Outcome,
    count(*) as Count
from SPACEXTABLE
where Date between "2010-06-04" and "2017-03-20"
group by Landing_Outcome
order by Count desc
```

```
* sqlite:///my_data1.db
Done.
```

Landing_Outcome	Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

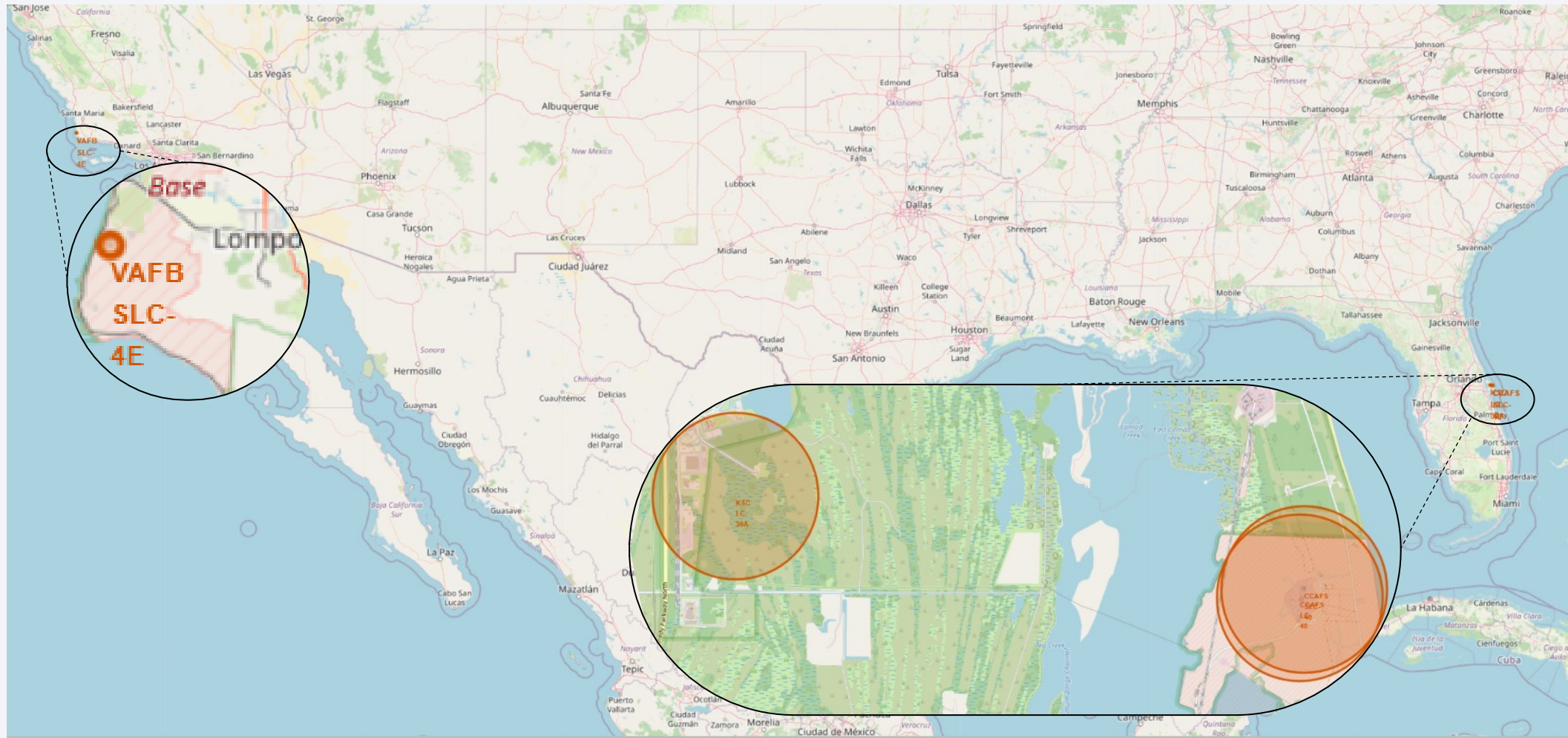
- A SELECT statement with COUNT function, WHERE clause using BETWEEN...AND operator, GROUP BY and ORDER BY clauses was utilized to identify records between June 4, 2010 and March 20, 2017 and show aggregated Landing Outcomes
- Landing Outcome categories were sorted by their respective counts in descending order



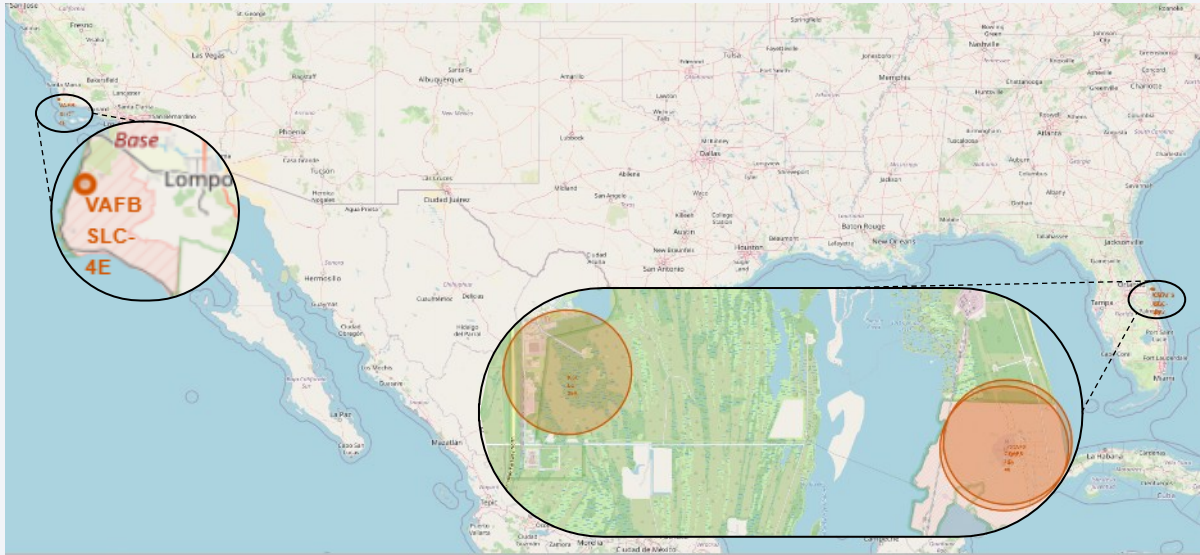
Section 3

Launch Sites Proximities Analysis

Launch Sites Locations



Launch Sites Locations

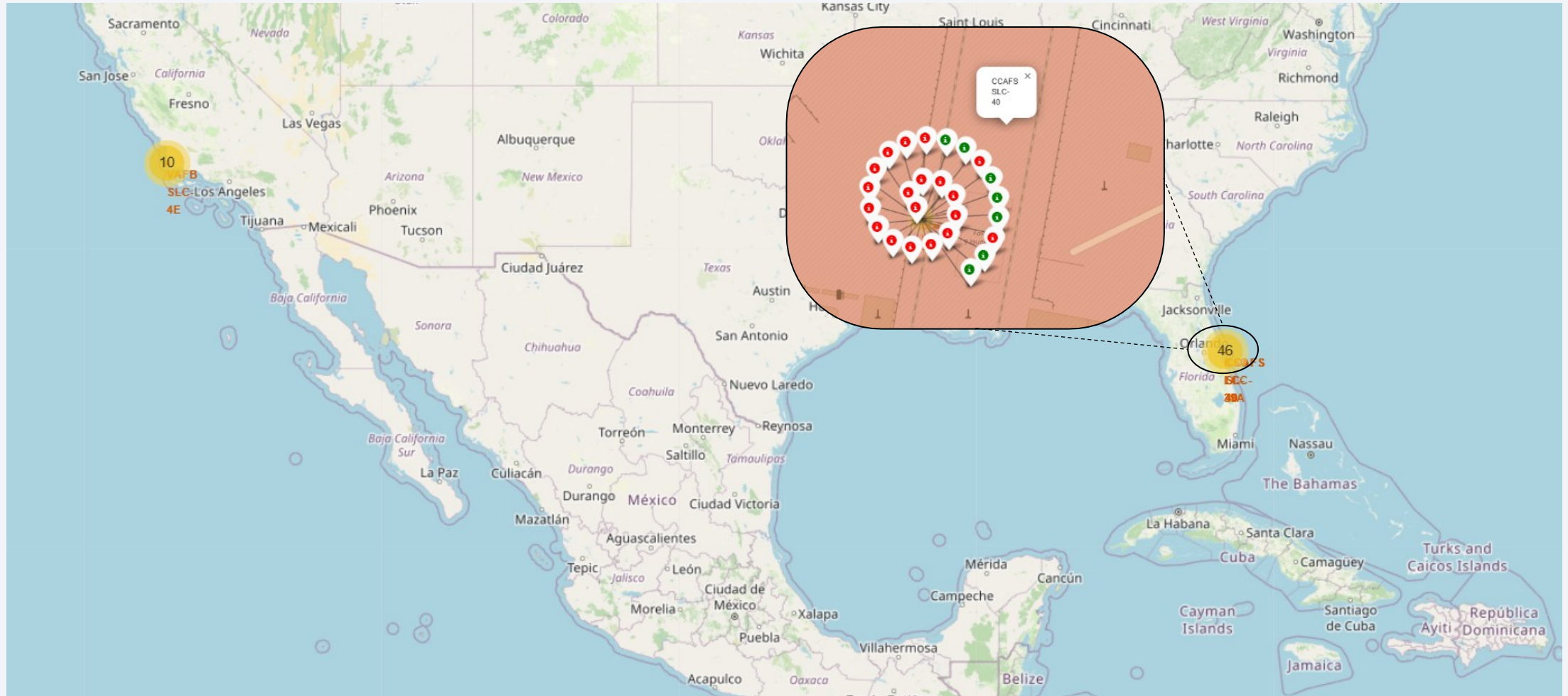


Launch Sites are highlighted with:

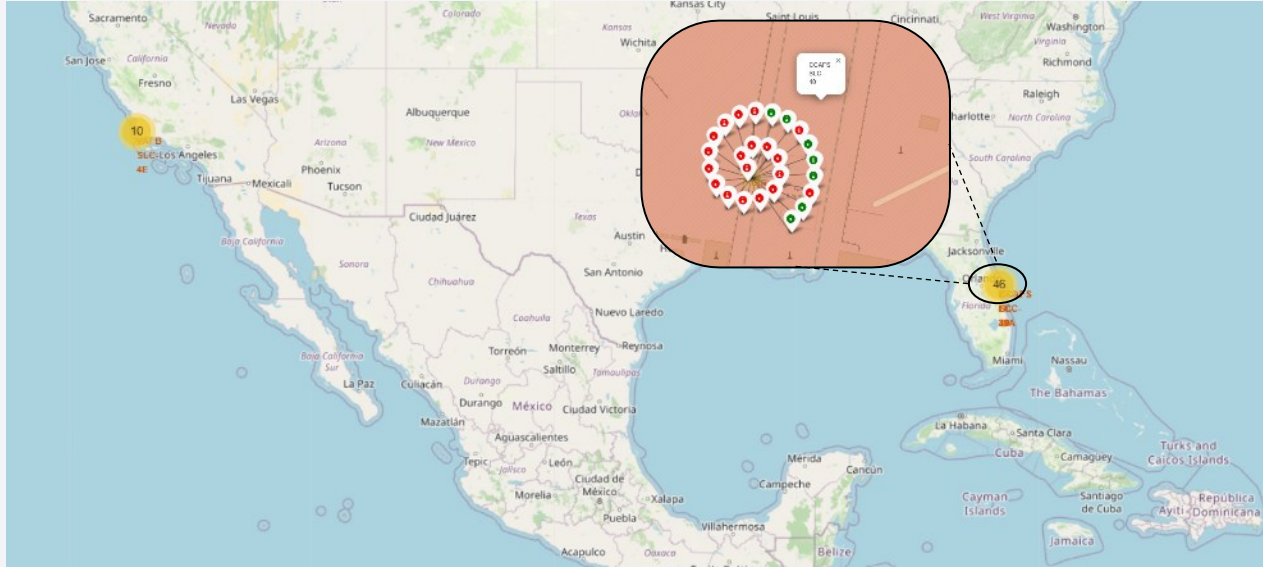
- Red Circles (1km radius) with pop-up (site name)
- Red Markers with the site names

- Launch Sites are located close to the shore for safety reasons (rockets are launched towards the ocean to avoid inhabited areas)
- CCAFS LC-40 and CCAFS SLC-40 refer to the same Launch Site
- Most Launch sites (2/3) are located on the east coast to benefit the Eastward Boost (Earth's rotation)
- VAFB launch site is located on the west coast since it launches South to Polar Orbits

Launch Outcomes Success / Failure



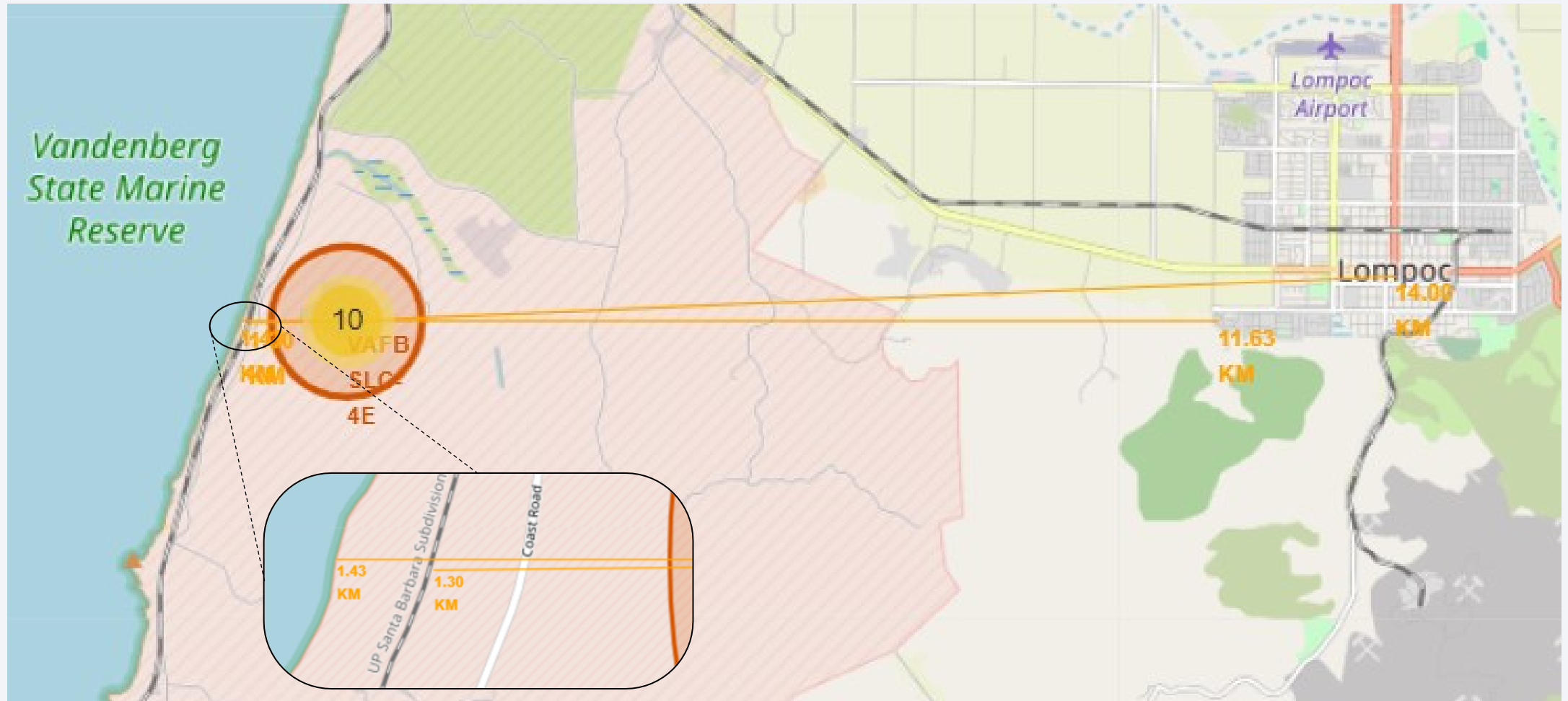
Launch Outcomes Success / Failure



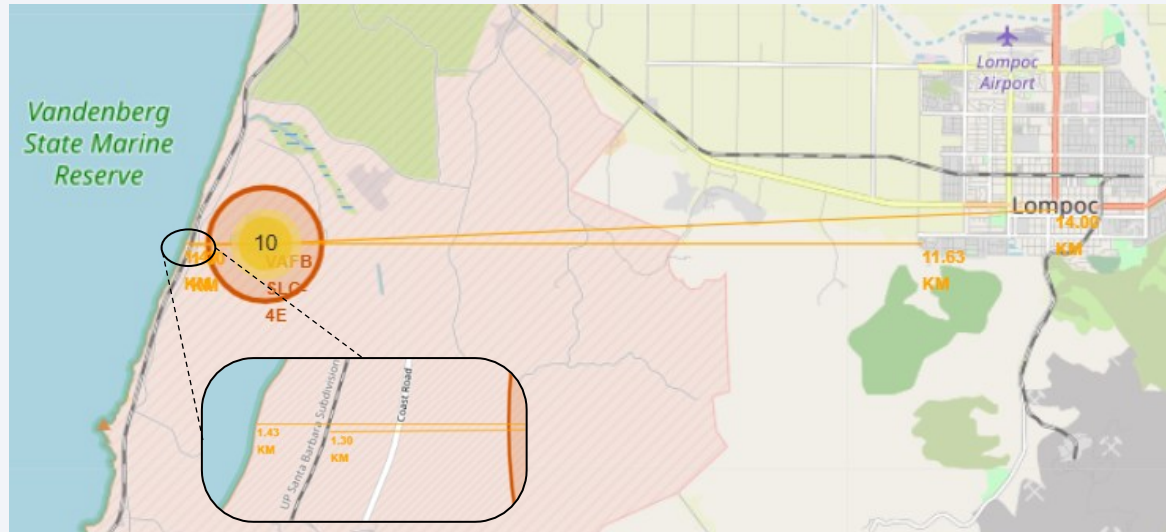
- For each launch, a marker was created (green – success, red - failure)
- Many markers share the same location. To improve visibility, markers have been added/grouped to dynamic MarkerCluster object

- Most launches are performed from the Launch Sites on the east coast
- The most frequented launch site is CCAFS SLC-40 with 26 lunches and the lowest success ratio of landing success

VAFB SLC-4E Launch Site Proximities



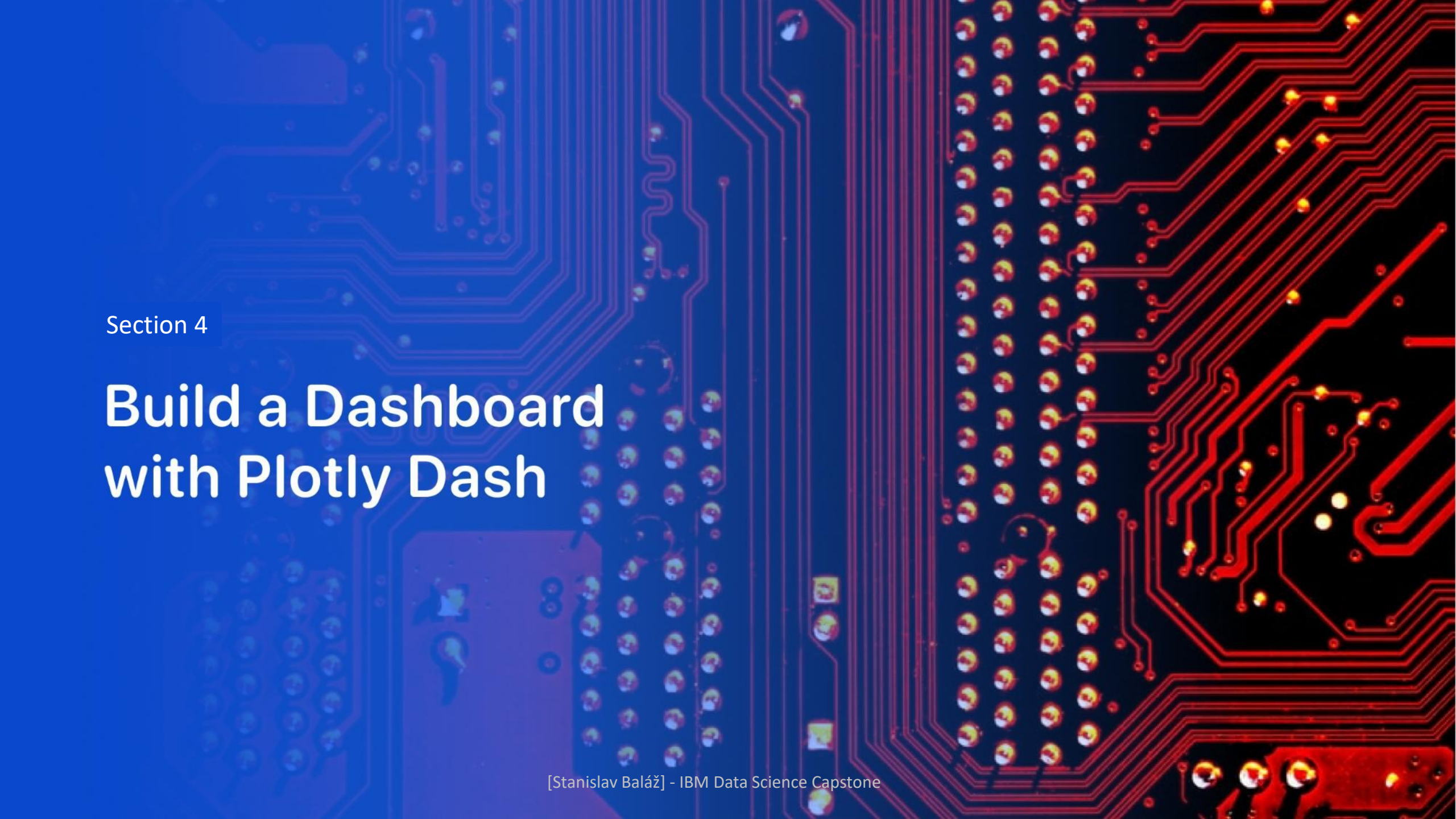
VAFB SLC-4E Launch Site Proximities



- For each Launch Site, markers with distance to its proximities (city, railway, highway) were created
- Lines (PolyLine) objects were drawn to highlight distances from a Launch Site to its proximities

Launch sites are located:

- Close to coast, so that rockets can be launched to the area above the ocean (safety)
- Close to the railway and/or highway, due to transport reasons
- Farther from cities (safety, elimination of discomfort due to noise and vibrations)



Section 4

Build a Dashboard with Plotly Dash

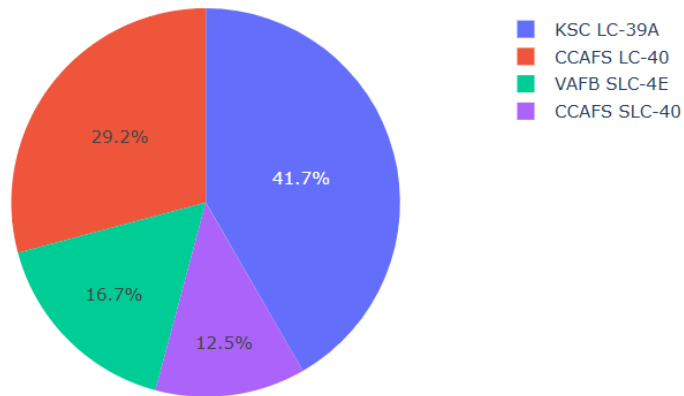
Total Success Launches by Site

SpaceX Launch Records Dashboard

All Sites



Total Success Launches by Site

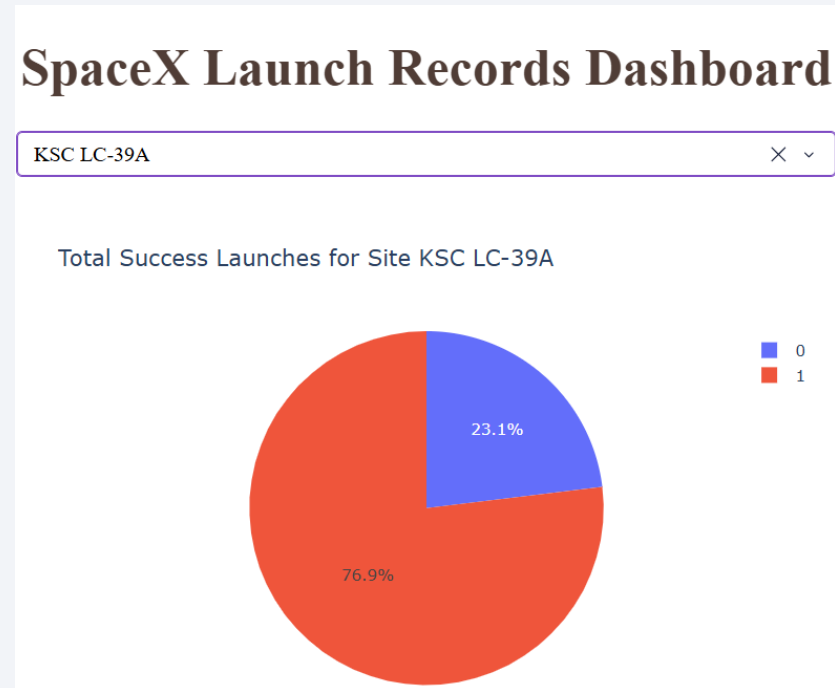


- Application contains a Dropdown box for Launch Site selection
- If All Sites are selected, then the pie chart (via Plotly Express) shows a summary of successful launches for all sites

- Out of the four launch sites, KSC LC-39A itself performed 41.7% of all successful launches
- The least portion of 12.5% was performed by CCAFS SLC-40

Site with The Highest Launch Success Ratio

KSC LC-39A



- Application contains a Dropdown box for Launch Site selected and a Pie Chart linked to the selection
- If an individual site is selected, then the Pie Chart (via Plotly Express) shows portions of success and failures for the launch site

- KSC LC-39A is the site with the highest launch success ratio of 76.9%
- As shown on the prior slide, this site has also the highest share of all successful launches

Payload vs. Launch Outcome vs. Booster Version



Payload vs. Launch Outcome vs. Booster Version



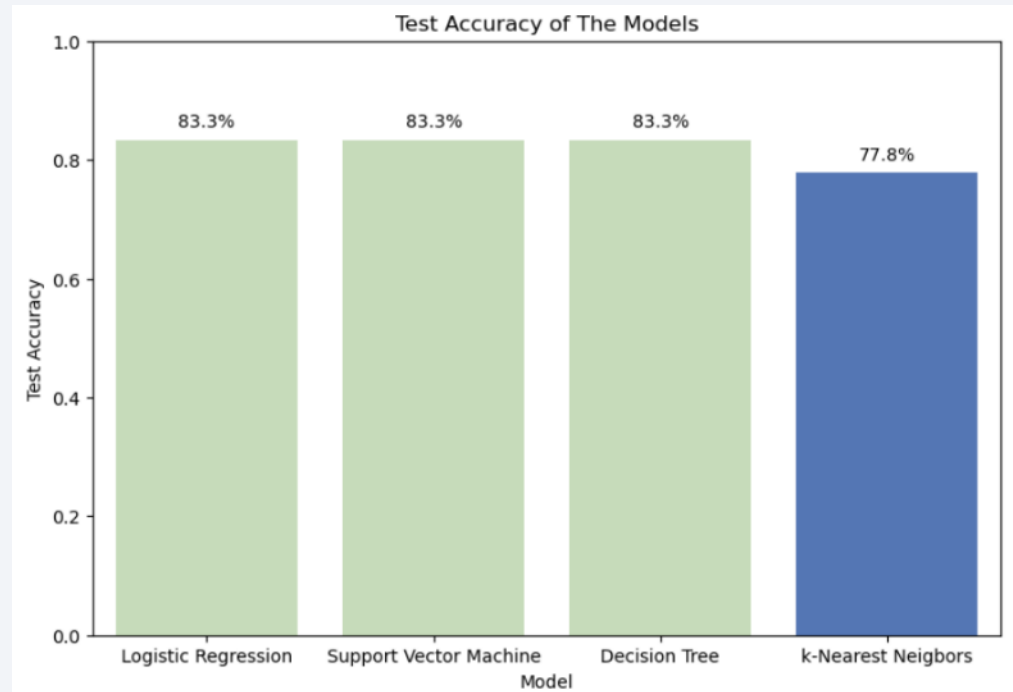
- Application contains a RangeSlider which allows us to select a Payload range to be analyzed, and a Scatter Plot linked to the selected Payload range
- The scatter plot shows individual launches for selected Payload range, split to success and failure, with color indicating a booster version

- Payload range 3000 – 4000 kg shows the highest launch success rate
- The lowest launch success rate is observed in payload range 6000 – 7000 kg
- FT Booster Version shows the highest launch success rate

Section 5

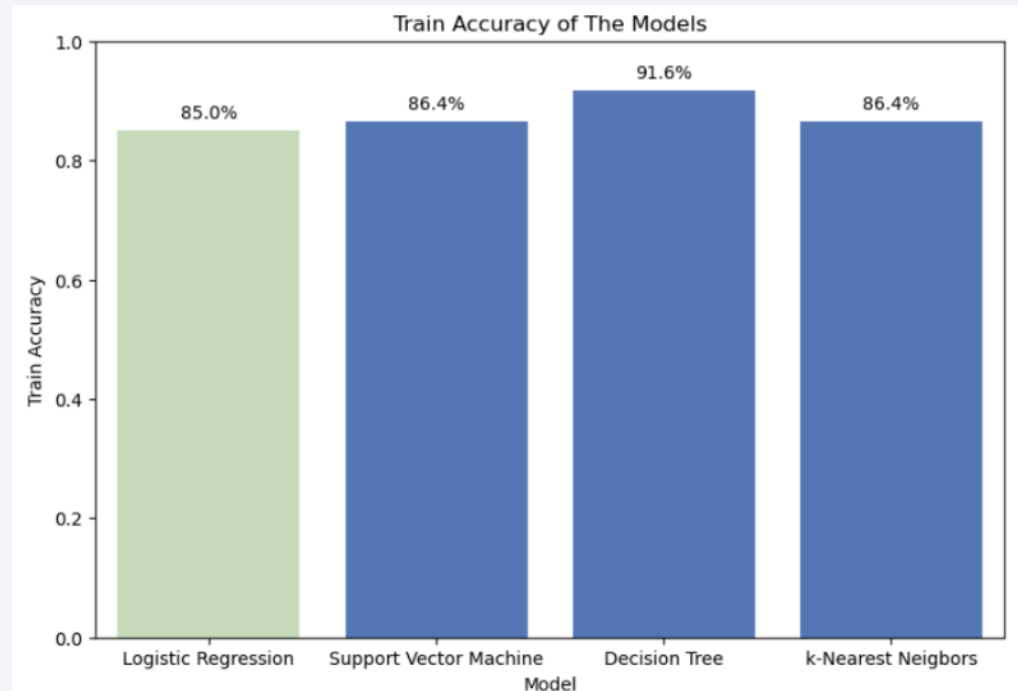
Predictive Analysis (Classification)

Classification Accuracy



- **Results:** Logistic Regression, Support Vector Machine, Decision Tree models achieved the highest Test Accuracy score of 83.3%
- **Secondary criterion:** Generalization Capacity was used to identify the best performing model
- Generalization Capacity indicates the model captures underlying pattern in training data rather than noise (model is less prone to overfitting)

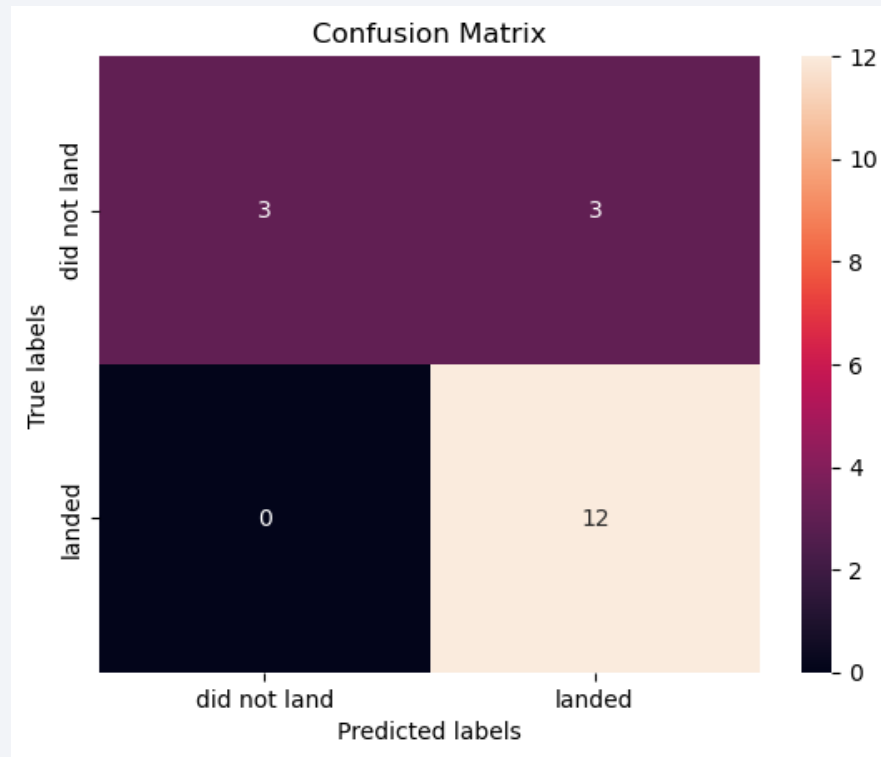
Classification Accuracy



- **Conclusion: Logistic Regression** model is selected as the best model since it has the smallest delta between testing and training accuracy
- The smallest gap indicates the best Generalization Capacity, i.e. model is less prone to overfitting

- The best parameters: 'C': 0.1, 'penalty': 'l2', 'solver': 'lbfgs'

Confusion Matrix



Confusion matrix of the Logistic Regression model exhibits:

- Impressive **Recall** of 100% (12 True Positives, and 0 False Negatives)
- **Accuracy** 83.3% (15 predictions from 18 are correct)
- **Precision** of 80% (12 True Positives, 15 Predicted Positives)
- **Specificity** (1-FPR) of 50% is a key model limitation (3 False Positives and 3 True Negatives)

Conclusions

Launch Sites Locations (*)

- Mainly on the east coast (2/3) – Eastward Boost
- Farther from cities and close to the coast, railway and highway

General

- Increasing trend in success rate from 2013 to 2020. Low success rate for the first 25 launches mainly performed at CCAFS SLC-40
- Increasing portion of heavy payloads (+10000 kg), except for VAFB SLC-4E

* (CCAFS LC-40 and CCAFS SLC-40 refer to the same Launch Site)

Orbits

- SSO, VLEO – orbits with the highest success rate (> 85%), having at least 5 launches
- VLEO – started from flight 65 – Starlink era. Heavy payloads +15000 kg.

Payloads

- The highest success rate payload range is 3000 – 4000 kg
- The lowest success rate payload range is 6000 – 7000 kg

Boosters

- FT Booster Version shows the highest launch success rate

Conclusions

- **Logistic Regression** was identified as the best performing classification model (Accuracy, Generalization Capacity) to answer the question “Will stage one land successfully?”
 - Solid results achieved for accuracy (83.3%), recall (100%), and precision (80%)
 - Specificity of 50% – model is too optimistic in predicting success. Significant tendency to false positives is the main area for improvement.
- **Action** – balance the model by improving Specificity – increase Decision Threshold to tune the model

Appendix A – Data Collection and Wrangling

Collection SpaceX API

- `static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/API_call_spacex_api.json'`
- `response = requests.get(static_json_url)`
- `data = pd.json_normalize(response.json())`
- `data_falcon9.to_csv('dataset_part_1.csv', index=False)`

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude
1	1	2010-06-04	Falcon 9	6123.547647058824	LEO	CCSFS SLC 40	None None	1	False	False	False		1.0	0	B0003	-80.577366	28.5618571
2	2	2012-05-22	Falcon 9	525.0	LEO	CCSFS SLC 40	None None	1	False	False	False		1.0	0	B0005	-80.577366	28.5618571
3	3	2013-03-01	Falcon 9	677.0	ISS	CCSFS SLC 40	None None	1	False	False	False		1.0	0	B0007	-80.577366	28.5618571
4	4	2013-09-29	Falcon 9	500.0	PO	VAFB SLC 4E	False Ocean	1	False	False	False		1.0	0	B1003	-120.610829	34.632093
5	5	2013-12-03	Falcon 9	3170.0	GTO	CCSFS SLC 40	None None	1	False	False	False		1.0	0	B1004	-80.577366	28.5618571
6	6	2014-01-06	Falcon 9	3325.0	GTO	CCSFS SLC 40	None None	1	False	False	False		1.0	0	B1005	-80.577366	28.5618571
7	7	2014-04-18	Falcon 9	2295.0	ISS	CCSFS SLC 40	True Ocean	1	False	False	True		1.0	0	B1006	-80.577366	28.5618571
8	8	2014-07-14	Falcon 9	1316.0	LEO	CCSFS SLC 40	True Ocean	1	False	False	True		1.0	0	B1007	-80.577366	28.5618571
9	9	2014-08-05	Falcon 9	4535.0	GTO	CCSFS SLC 40	None None	1	False	False	False		1.0	0	B1008	-80.577366	28.5618571
10	10	2014-09-07	Falcon 9	4428.0	GTO	CCSFS SLC 40	None None	1	False	False	False		1.0	0	B1011	-80.577366	28.5618571
11	11	2014-09-21	Falcon 9	2216.0	ISS	CCSFS SLC 40	False Ocean	1	False	False	False		1.0	0	B1010	-80.577366	28.5618571
12	12	2015-01-10	Falcon 9	2395.0	ISS	CCSFS SLC 40	False ASDS	1	True	False	True	761634e7cb	1.0	0	B1012	-80.577366	28.5618571

Collection Wikipedia

- `session.get("https://en.wikipedia.org/", headers=headers)response = session.get(static_url, headers=headers)response.status_code`
- `html = response.text`
- `soup = BeautifulSoup(html, "html.parser")`
- `df.to_csv('spacex_web_scraped.csv', index=False)`

	Flight No.	Launch site	Payload	Payload mass	Orbit	Customer	Launch outcome	Version Booster	Booster landing	Date	Time
1	1	CCAFS	acecraft Qualification Unit	0	LEO	SpaceX	Success	F9 v1.07B0003.18	Failure	4 June 2010	18:45
2	2	CCAFS	Dragon	0	LEO	NASA	Success	F9 v1.07B0004.18	Failure	8 December 2010	15:43
3	3	CCAFS	Dragon	525 kg	LEO	NASA	Success	F9 v1.07B0005.18	No attempt	22 May 2012	07:44
4	4	CCAFS	SpaceX CRS-1	4,700 kg	LEO	NASA	Success	F9 v1.07B0006.18	No attempt	8 October 2012	00:35
5	5	CCAFS	SpaceX CRS-2	4,877 kg	LEO	NASA	Success	F9 v1.07B0007.18	No attempt	1 March 2013	15:10
6	6	VAFB	CASSIOPE	500 kg	Polar orbit	MDA	Success	F9 v1.17B10038	Uncontrolled	29 September 2013	16:00
7	7	CCAFS	SES-8	3,170 kg	GTO	SES	Success	F9 v1.1	No attempt	3 December 2013	22:41
8	8	CCAFS	Thaicom 6	3,325 kg	GTO	Thaicom	Success	F9 v1.1	No attempt	6 January 2014	22:06
9	9	Cape Canaveral	SpaceX CRS-3	2,296 kg	LEO	NASA	Success	F9 v1.1	Controlled	18 April 2014	19:25
10	10	Cape Canaveral	Orbcomm-OG2	1,316 kg	LEO	Orbcomm	Success	F9 v1.1	Controlled	14 July 2014	15:15

Wrangling

- `PayloadMass_mean = data_falcon9['PayloadMass'].mean()`
- `data_falcon9['PayloadMass'] = data_falcon9['PayloadMass'].fillna(PayloadMass_mean)`
- `bad_outcomes=set(landing_outcomes.keys()[[1,3,5,6,7]])`
- `landing_class = [int(Out not in bad_outcomes) for Out in df['Outcome']]`

`df.head(5)`

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude	Class
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.577366	28.561857	0
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.577366	28.561857	0
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.577366	28.561857	0
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.610829	34.632093	0
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.577366	28.561857	0

Appendix B – EDA SQL & Visualization

SQL

- `%sql create table SPACEXTABLE as select * from SPACEXTBL where Date is not null`
- `select avg(PAYLOAD_MASS_KG_) as Average_Payload_Mass from SPACEXTABLE where "Booster_Version" like "F9 v1.1%"`
- `select substr("Mission_Outcome", 1, 7) as Mission_Outcome, count(*) as Count from SPACEXTABLE group by substr("Mission_Outcome", 1, 7)`
- `select Booster_Version, PAYLOAD_MASS_KG_ from SPACEXTABLE where PAYLOAD_MASS_KG_ = (select max(PAYLOAD_MASS_KG_) from SPACEXTABLE) order by Booster_Version asc`
- `select Landing_Outcome, count(*) as Count from SPACEXTABLE where Date between "2010-06-04" and "2017-03-20" group by Landing_Outcome order by Count desc`

Average_Payload_Mass
2534.6666666666665

Mission_Outcome	Count
Failure	1
Success	100

Booster_Version	PAYLOAD_MASS_KG_
F9 B5 B1048.4	15600
F9 B5 B1048.5	15600
F9 B5 B1049.4	15600
F9 B5 B1049.5	15600
F9 B5 B1049.7	15600
F9 B5 B1051.3	15600
F9 B5 B1051.4	15600
F9 B5 B1051.6	15600
F9 B5 B1056.4	15600
F9 B5 B1058.3	15600
F9 B5 B1060.2	15600
F9 B5 B1060.3	15600

Landing_Outcome	Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

Visualization

Flight Number and Launch Site

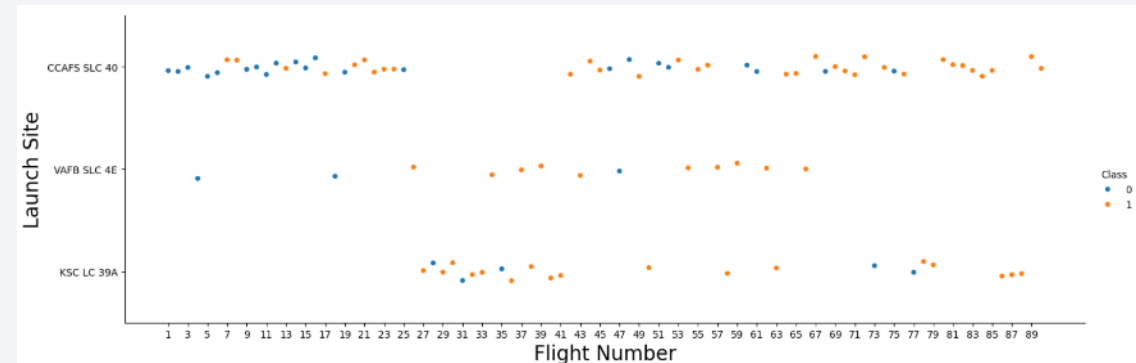
```
sns.catplot(y="LaunchSite", x="FlightNumber", hue="Class", data=df, aspect=3)
```

```
plt.xlabel("Flight Number", fontsize=20)
```

```
plt.ylabel("Launch Site", fontsize=20)
```

```
plt.xticks(ticks=range(1, 91, 2))
```

```
plt.show()
```



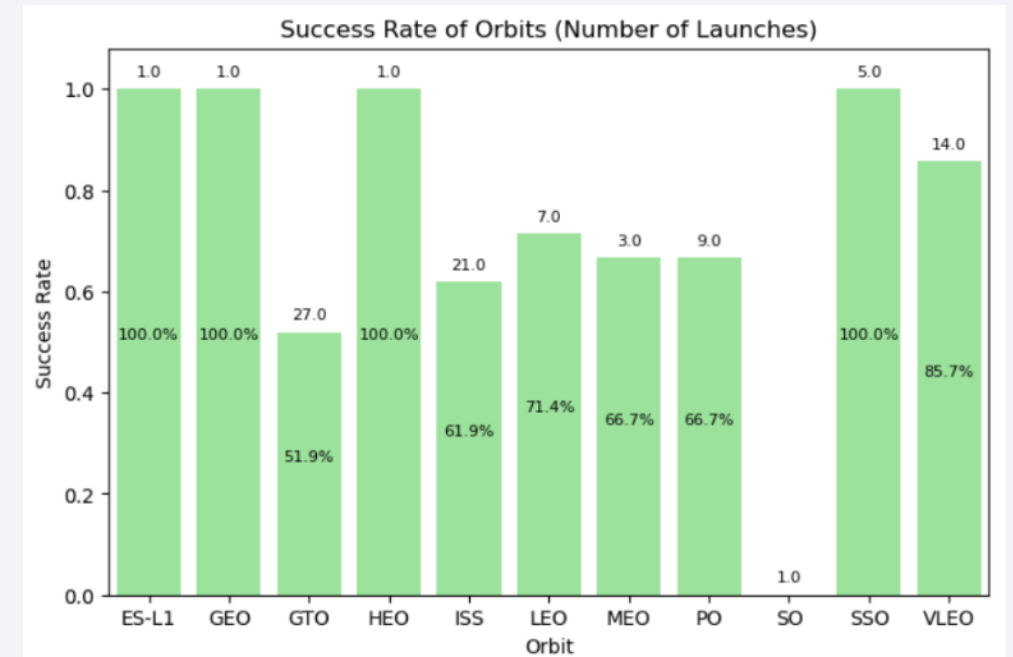
Appendix B – EDA SQL & Visualization

Visualization

Orbits Success Rate

```
orbit_success = df.groupby('Orbit')['Class'].agg(['mean', 'count'])
orbit_success.columns=['Success_Rate', 'Count']
plt.figure(figsize=(8, 5))
ax = sns.barplot(x='Orbit', y='Success_Rate', data=orbit_success, color='lightgreen')
for i, row in orbit_success.iterrows():
    ax.text(i, row['Success_Rate'] + 0.02, str(row['Count']), ha='center', va='bottom',
            fontsize=8)
    if row['Success_Rate']:
        ax.text(i, row['Success_Rate'] * 0.5, f"{row['Success_Rate']:.1%}", ha='center',
                va='bottom', fontsize=8)

plt.ylim(0, 1.08)
plt.xlabel('Orbit')
plt.ylabel('Success Rate')
plt.title('Success Rate of Orbits (Number of Launches)')
plt.show()
```



Appendix B – EDA SQL & Visualization

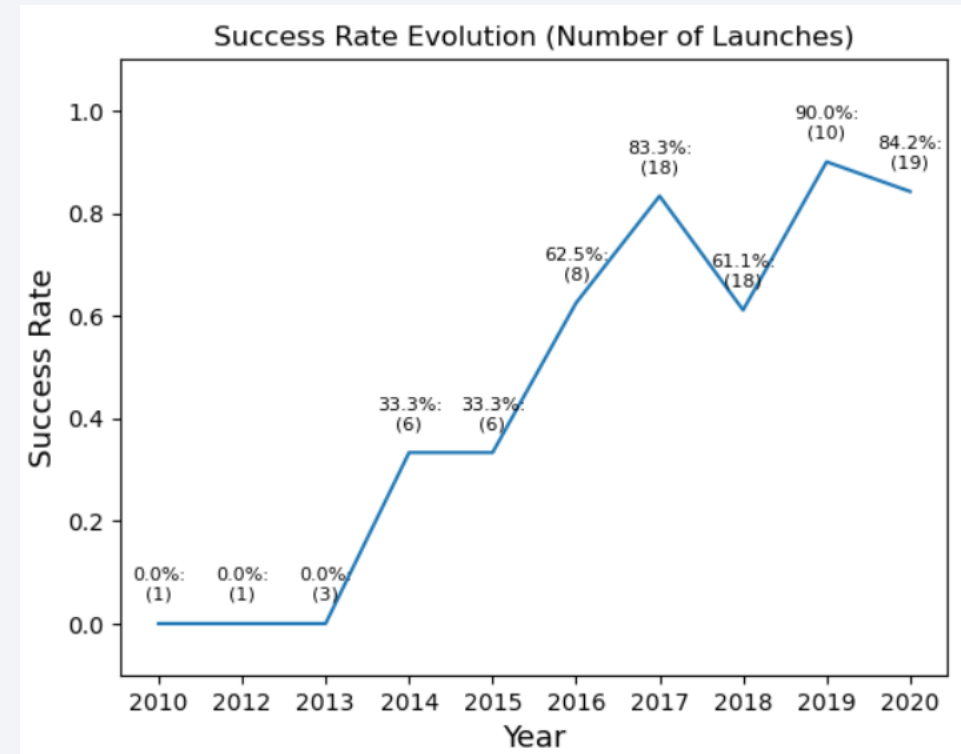
Visualization

Success Rate Evolution

```
year_success = df.groupby('Year')['Class'].agg(['mean', 'count'])
year_success.columns=['Success_Rate', 'Count']
year_success.reset_index(inplace=True)

ax = sns.lineplot(x='Year', y='Success_Rate', data=year_success)
for i, row in year_success.iterrows():
    ax.text(row['Year'], row['Success_Rate'] + 0.04,
            f"{row['Success_Rate']:.1%}:\n({str(row['Count'])})",
            ha='center', va='bottom', fontsize=8)

plt.ylim(-0.1, 1.1)
plt.xlabel("Year", fontsize=13)
plt.ylabel("Success Rate", fontsize=13)
plt.title("Success Rate Evolution (Number of Launches)")
plt.show()
```



Appendix C – Interactive Visual Analytics

Folium - Geospatial Interactive Analytics

```
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)

for i, site in launch_sites_df.iterrows():

    site_name = site['Launch_Site']

    site_coordinate = [site['Lat'], site['Long']]

    circle_site = folium.Circle(site_coordinate, radius=1000,

                                color='#d35400', fill=True).add_child(folium.Popup(site_name))

    marker_site = folium.map.Marker(site_coordinate,

                                     icon=DivIcon(icon_size=(20,20), icon_anchor=(0,-3),

                                                  html='<div style="font-size: 12; color:#d35400;"><b>%s</b></div>' %

                                                  site_name,

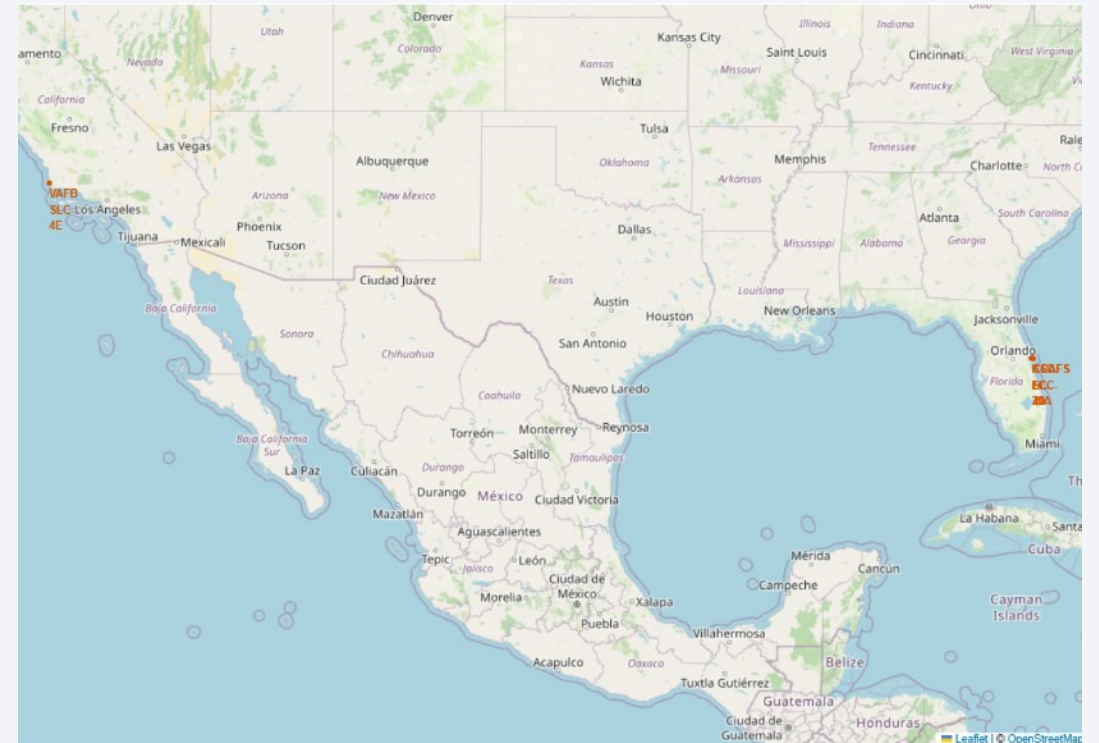
                                                  ),

                                     ),

    site_map.add_child(circle_site)

    site_map.add_child(marker_site)

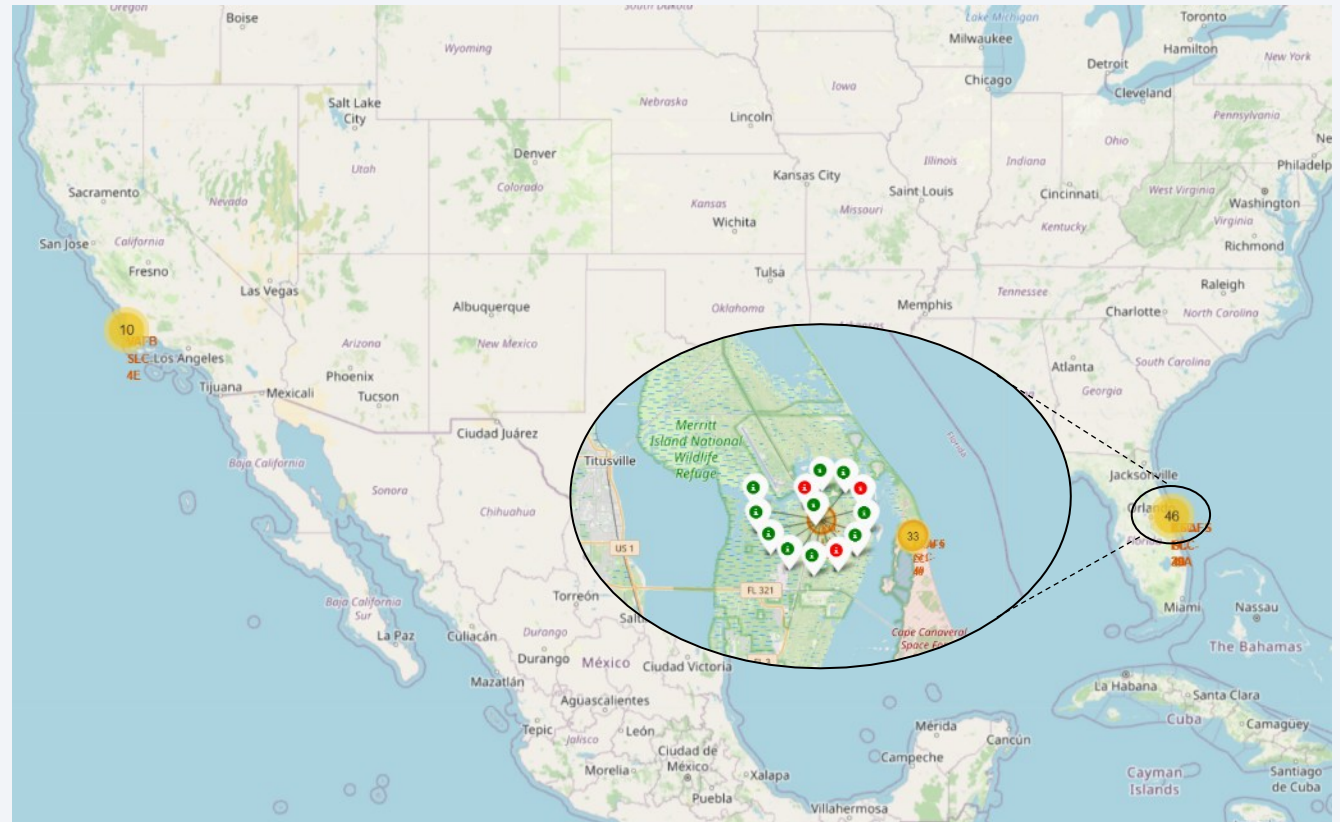
site_map
```



Appendix C – Interactive Visual Analytics

Folium - Geospatial Interactive Analytics

```
marker_cluster = MarkerCluster()
for index, row in spacex_df.iterrows():
    marker = folium.Marker(
        location=[row['Lat'], row['Long']],
        icon=folium.Icon(
            color='white',
            icon_color=row['marker_color']
        )
    )
    marker_cluster.add_child(marker)
site_map
```



Appendix C – Interactive Visual Analytics

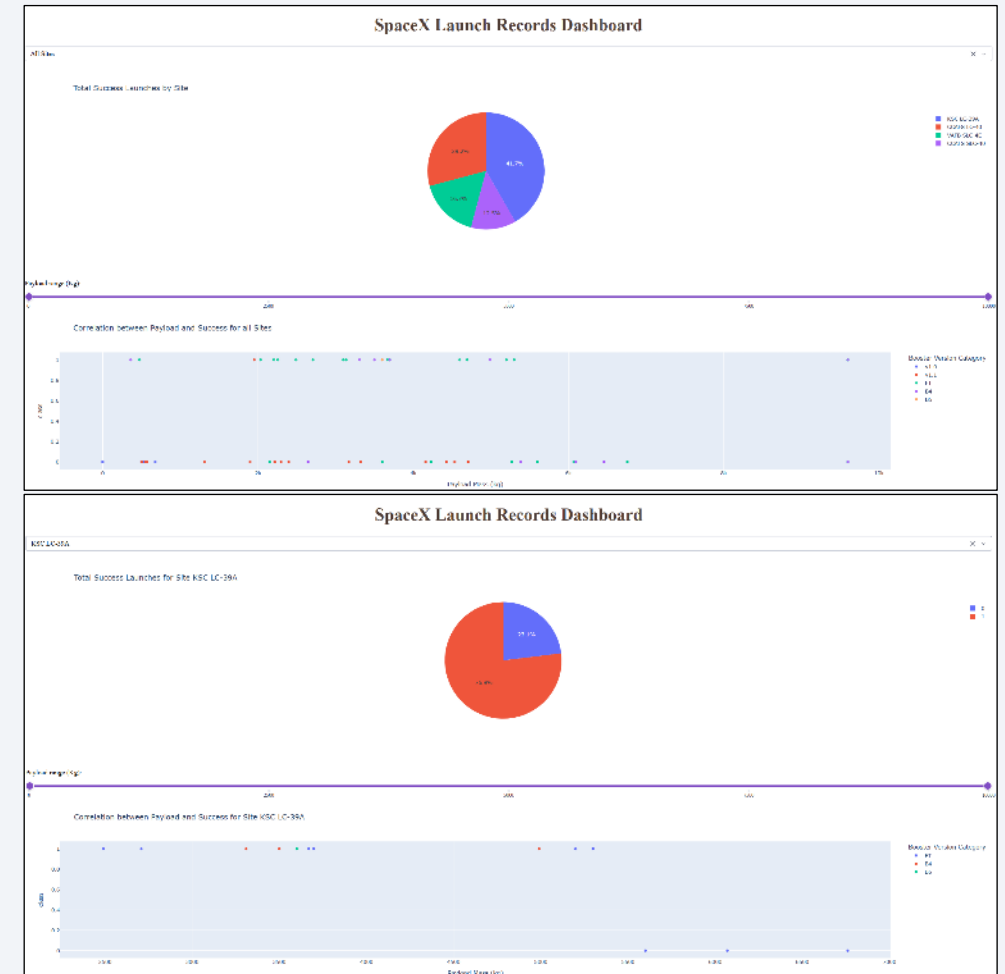
Plotly Dash - Interactive Launch Performance Dashboard

```
@app.callback(
    Output(component_id='success-payload-scatter-chart', component_property='figure'),
    [
        Input(component_id='site-dropdown', component_property='value'),
        Input(component_id="payload-slider", component_property="value")
    ]
)

def get_scatter_chart(entered_site, payload_range):
    filtered_df = spacex_df[spacex_df['Payload Mass (kg)'].between(*payload_range,
        inclusive='both')]

    # All Launch Sites
    if entered_site == 'ALL':
        fig = px.scatter(
            filtered_df,
            x='Payload Mass (kg)',
            y='class',
            color='Booster Version Category',
            title='Correlation between Payload and Success for all Sites'
        )
        return fig

    # Selected Launch Site
    else:
        filtered_df = filtered_df[filtered_df['Launch Site'] == entered_site]
        fig = px.scatter(
            filtered_df,
            x='Payload Mass (kg)',
            y='class',
            color='Booster Version Category',
            title='Correlation between Payload and Success for Site ' + entered_site
        )
        return fig
```



Appendix D – Machine Learning Details

Features Engineering - One-Hot Encoding

```
features = df[['FlightNumber', 'PayloadMass', 'Orbit', 'LaunchSite', 'Flights',  
'GridFins', 'Reused', 'Legs', 'LandingPad', 'Block', 'ReusedCount', 'Serial']]
```

```
features_one_hot = pd.get_dummies(features, columns=['Orbit', 'LaunchSite',  
'LandingPad', 'Serial'], drop_first=False)
```

```
features_one_hot = features_one_hot.astype('float64')
```

	FlightNumber	PayloadMass	Flights	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	Orbit_GTO	Orbit_HEO	Orbit_ISS	...	Serial_B1058	Serial_B1059	Serial_B1060	Serial_B1062	GridFins_False	GridFins_True	Reused_False	Reused_True	Legs_False	Legs_True
0	1.0	6104.959412	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0
1	2.0	525.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0
2	3.0	677.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0
3	4.0	500.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0
4	5.0	3170.000000	1.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
85	86.0	15400.000000	2.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0
86	87.0	15400.000000	3.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0
87	88.0	15400.000000	6.0	5.0	5.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0
88	89.0	15400.000000	3.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0
89	90.0	3681.000000	1.0	5.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	0.0	1.0

90 rows × 83 columns

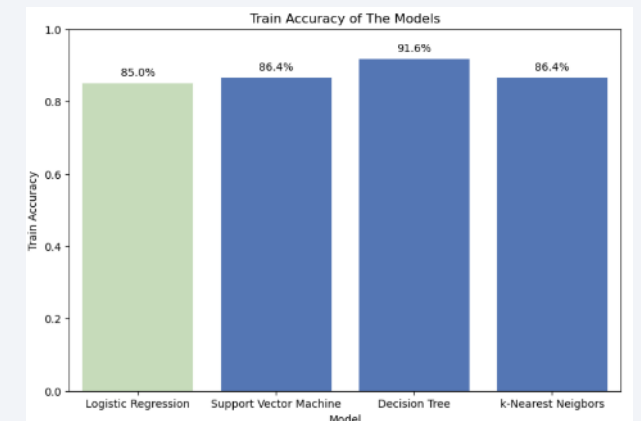
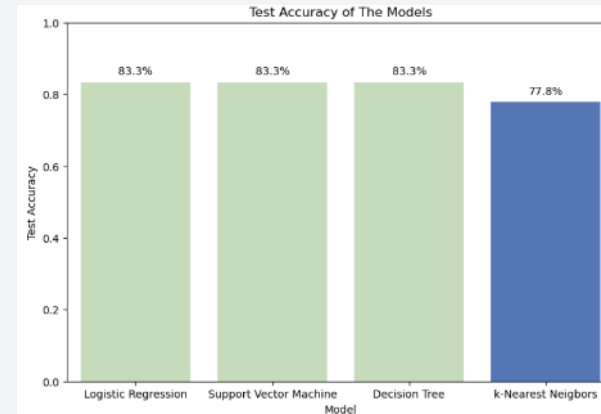
Appendix D – Machine Learning Details

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 6))
green = '#C5E0B4'
blue = '#4472C4'
padding = 5

colors_test = [green if x==max_Test_Accuracy else blue for x in results['Test_Accuracy']]
sns.barplot(x='Model', y='Test_Accuracy', data=results, hue='Model', palette=colors_test, ax=ax1)
ax1.set_ylim(0,1)
ax1.set_ylabel('Test Accuracy')
ax1.set_title('Test Accuracy of The Models')
for container in ax1.containers:
    labels=[f'{value:.1%}' for value in container.datavalues]
    ax1.bar_label(container, labels=labels, padding=padding)

colors_train = [green, blue, blue, blue]
sns.barplot(x='Model', y='Train_Accuracy', data=results, hue='Model', palette=colors_train, ax=ax2)
ax2.set_ylim(0,1)
ax2.set_ylabel('Train Accuracy')
ax2.set_title('Train Accuracy of The Models')
for container in ax2.containers:
    labels=[f'{value:.1%}' for value in container.datavalues]
    ax2.bar_label(container, labels=labels, padding=padding)

plt.show()
```



Appendix D – Machine Learning Details

Logistic Regression – The Best Model

```
parameters = {"C":[0.01, 0.1,  
1], 'penalty':['l2'], 'solver':['lbfgs']}
```

```
lr = LogisticRegression()
```

```
logreg_cv = GridSearchCV(estimator=lr,  
param_grid=parameters, cv=10, verbose=1)
```

```
logreg_cv.fit(X_train, Y_train)
```

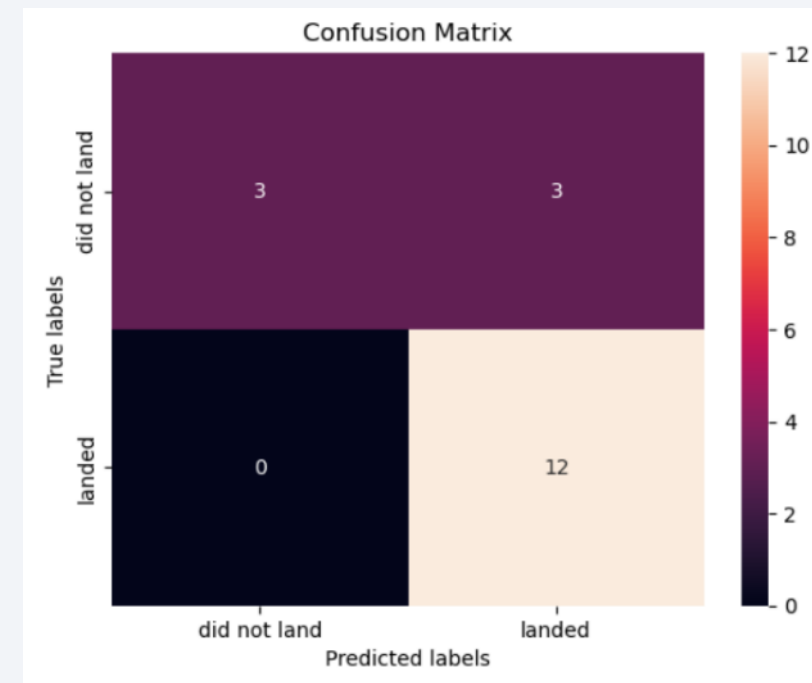
```
logreg_cv.score(X_test, Y_test)
```

```
yhat=logreg_cv.predict(X_test)
```

```
plot_confusion_matrix(Y_test, yhat)
```

```
logreg_cv.score(X_test, Y_test)
```

```
0.8333333333333334
```



Thank you!

